

Technical Specifications & Low-Level Design (TTD_LLD)

Table of Contents

1	Technical Specifications & Low-Level Design (TTD/LLD)	5
1.1	TakeLow — Lowest Unique Bid Auction Platform	5
1.2	Document Control	5
1.2.1	Version History	5
1.2.2	Document Metadata	5
1.2.3	Distribution List.....	6
1.2.4	Confidentiality Notice.....	6
1.3	Table of Contents	6
1.4	1. Technology Stack	7
1.4.1	1.1 Backend Services	7
1.4.2	1.2 Frontend Applications.....	8
1.4.3	1.3 Infrastructure	8
1.4.4	1.4 Caching Stack	9
1.5	2. Database Schema (ERD)	9
1.5.1	2.1 Entity-Relationship Diagram.....	9
1.5.2	2.2 Table Definitions.....	10
1.5.3	2.3 Prisma Schema (Excerpt)	18
1.5.4	2.4 Database Index Catalog.....	22
1.5.5	2.5 Database Functions and Sequences.....	25
1.6	3. API Contracts	25
1.6.1	3.1 API Overview	25
1.6.2	3.2 Identity Service API	26
1.6.3	3.3 Auction Engine API.....	33
1.6.4	3.4 Query Service API	39
1.6.5	3.5 Proxy Configurations.....	44
1.7	4. Data Transfer Objects (DTOs)	45
1.7.1	4.1 Identity Service DTOs.....	45

1.7.2	4.2 Auction Engine DTOs	46
1.7.3	4.3 Query Service DTOs	48
1.8	5. Error Handling and Retry Logic	50
1.8.1	5.1 Error Response Format	50
1.8.2	5.2 Error Codes	50
1.8.3	5.3 Error Handling Matrix	52
1.8.4	5.4 Retry Logic	52
1.8.5	5.5 Circuit Breaker Pattern.....	53
1.8.6	5.6 Rate Limiting Configuration	55
1.8.7	5.7 Caching Strategy	55
1.9	6. Versioning Strategy	57
1.9.1	6.1 API Versioning	57
1.9.2	6.2 Database Migration Versioning.....	57
1.9.3	6.3 Frontend Versioning.....	57
1.10	7. Integration Points	57
1.10.1	7.1 ERP/Oracle Integration	57
1.10.2	7.2 Payment Gateway Integration.....	59
1.10.3	7.3 SMS Integration	59
1.10.4	7.4 Push Notification Integration.....	60
1.10.5	7.5 Social Login Integration.....	60
1.11	8. Performance Benchmarks	60
1.11.1	8.1 Target Performance	60
1.11.2	8.2 Database Query Performance	60
1.11.3	8.3 Load Testing Scenarios	61
1.12	9. Coding Standards.....	61
1.12.1	9.1 Backend (NestJS).....	61
1.12.2	9.2 Frontend (React).....	61
1.12.3	9.3 Security Standards	62
1.12.4	9.4 Git Conventions.....	62
1.13	10. Redis HA Implementation	62
1.13.1	10.1 Redis Provider Factory	62
1.13.2	10.2 Environment Configuration.....	63
1.13.3	10.3 Docker Compose Files	63

1.13.4	10.4 Kubernetes Manifests	64
1.14	11. Audit Logging Implementation	64
1.14.1	11.1 Audit Module Structure	64
1.14.2	11.2 Audit Log Prisma Model.....	64
1.14.3	11.3 Audit Service Interface	65
1.14.4	11.4 Integration Points.....	65
1.14.5	11.5 Immutability Enforcement.....	67
1.15	12. Backup/Restore Scripts	67
1.15.1	12.1 pgBackRest Backup Script	67
1.15.2	12.2 WAL-G Backup Script.....	67
1.15.3	12.3 Disaster Recovery Restore Script.....	68
1.15.4	12.4 Kubernetes CronJobs	68
1.16	13. WebSocket Redis Adapter.....	69
1.16.1	13.1 Implementation	69
1.16.2	13.2 Cross-Instance Message Flow	69
1.16.3	13.3 Environment Configuration.....	70
1.16.4	13.4 Kubernetes Horizontal Scaling.....	70
1.17	14. Test Strategy & Coverage.....	71
1.17.1	14.1 Testing Pyramid	71
1.17.2	14.2 Unit Testing	71
1.17.3	14.3 Integration Testing	72
1.17.4	14.4 End-to-End (E2E) Testing	72
1.17.5	14.5 Performance Testing	72
1.17.6	14.6 Security Testing	73
1.17.7	14.7 Coverage Reporting	73
1.18	15. Code Quality Standards	73
1.18.1	15.1 Linting	73
1.18.2	15.2 Formatting.....	74
1.18.3	15.3 Complexity Metrics	74
1.18.4	15.4 Code Review Process.....	75
1.18.5	15.5 Pre-commit & Pre-merge Hooks	75
1.19	16. API Versioning & Backward Compatibility.....	75
1.19.1	16.1 Versioning Strategy	75

1.19.2	16.2 Backward Compatibility Rules	76
1.19.3	16.3 Deprecation Policy.....	76
1.19.4	16.4 Compatibility Testing	77
1.20	17. Database Migration Strategy	77
1.20.1	17.1 Prisma Migration Workflow.....	77
1.20.2	17.2 Migration File Structure	78
1.20.3	17.3 Zero-Downtime Deployment Procedure	78
1.20.4	17.4 Backward-Compatible Migration Rules	78
1.20.5	17.5 Rollback Procedures	79
1.20.6	17.6 Migration Testing	79
1.21	18. Configuration Management.....	79
1.21.1	18.1 Environment Management	79
1.21.2	18.2 Environment Variable Catalog	80
1.21.3	18.3 Secret Management.....	81
1.21.4	18.4 Feature Flags.....	81
1.22	19. Observability & Monitoring	81
1.22.1	19.1 Logging.....	81
1.22.2	19.2 Metrics	82
1.22.3	19.3 Distributed Tracing.....	83
1.22.4	19.4 Alerting.....	83
1.22.5	19.5 Dashboards.....	84
1.23	20. CI/CD Pipeline Details	85
1.23.1	20.1 Pipeline Overview	85
1.23.2	20.2 Pipeline Stages	85
1.23.3	20.3 GitHub Actions Workflow (Excerpt)	86
1.23.4	20.4 Deployment Strategies.....	88
1.23.5	20.5 Pipeline Gates & Quality Enforcement	88

1 Technical Specifications & Low-Level Design (TTD/LLD)

1.1 TakeLow — Lowest Unique Bid Auction Platform

Document Title: Technical Specifications & Low-Level Design (TTD/LLD) **Product:** TakeLow — Lowest Unique Bid Auction Platform **Version:** 3.0 **Date:** 2026-08-18 **Status:** Approved for Submission **Classification:** Confidential — Internal & Authorized Reviewers Only

1.2 Document Control

1.2.1 Version History

Version	Date	Author	Reviewer	Approver	Summary of Changes
1.0	2026-06-10	Engineering Team	Tech Lead	CTO	Initial TTD/LLD draft
2.0	2026-07-22	Engineering Team	Architecture Review Board	CTO	Added Redis HA, audit logging, backup/restore, WebSocket adapter
3.0	2026-08-18	Engineering Team	Architecture Review Board	CTO	Enterprise hardening: Prisma ORM, Better-auth, Redis caching, test strategy, observability, CI/CD, migration strategy, enhanced API contracts, error matrix, circuit breaker, rate limiting, caching strategy

1.2.2 Document Metadata

Field	Value
Document ID	TAK-TTD-LLD-003
Version	3.0
Status	Approved for Submission
Classification	Confidential
Author	Engineering Team
Reviewer	Architecture Review Board
Approver	Chief Technology Officer

Field	Value
Issue Date	2026-08-18
Next Review Date	2027-02-18
Document Owner	Engineering Team

1.2.3 Distribution List

Recipient / Role	Organization	Access Level
Architecture Review Board	TakeLow Engineering	Full
Chief Technology Officer	TakeLow Executive	Full
VP of Engineering	TakeLow Engineering	Full
Lead Security Architect	TakeLow Security	Full
Lead DevOps / SRE	TakeLow Infrastructure	Full
QA Engineering Lead	TakeLow QA	Full
Database Administrator	TakeLow Infrastructure	Full
External Technical Assessor	Designated Review Authority	Read-only

1.2.4 Confidentiality Notice

This document contains proprietary and confidential information belonging to TakeLow. It is provided solely for the purpose of technical review and assessment by authorized recipients. This document and its contents may not be reproduced, distributed, disclosed, or used for any purpose other than the stated review without the prior written consent of TakeLow. Unauthorized use, duplication, or disclosure is strictly prohibited. All rights reserved.

1.3 Table of Contents

1. Technology Stack
2. Database Schema (ERD)
3. API Contracts
4. Data Transfer Objects (DTOs)
5. Error Handling and Retry Logic
6. Versioning Strategy
7. Integration Points
8. Performance Benchmarks
9. Coding Standards
10. Redis HA Implementation
11. Audit Logging Implementation

- 12. Backup/Restore Scripts
- 13. WebSocket Redis Adapter
- 14. Test Strategy & Coverage
- 15. Code Quality Standards
- 16. API Versioning & Backward Compatibility
- 17. Database Migration Strategy
- 18. Configuration Management
- 19. Observability & Monitoring
- 20. CI/CD Pipeline Details

1.4 1. Technology Stack

1.4.1 1.1 Backend Services

Component	Technology	Version	Purpose
Runtime	Node.js	20.x	JavaScript runtime
Framework	NestJS	10.x	Server-side application framework
Language	TypeScript	5.x	Type-safe JavaScript
ORM	Prisma	5.x	Database object-relational mapping, type-safe query builder, declarative schema migrations
Database	PostgreSQL	15.x	Primary data store
Extension	Citus	12.1	Distributed PostgreSQL
Cache	Redis	7.x	In-memory data store, caching, rate limiting, pub/sub
Queue	BullMQ	5.x	Redis-based job queue
Real-time	Socket.io	4.x	WebSocket communication
Authentication	Better-auth +	—	Authentication

Component	Technology	Version	Purpose
	JwtAuthGuard		framework with JWT guard, session management, and pluggable OAuth strategies
Validation	class-validator	—	DTO validation
Cryptography	Node.js crypto	built-in	AES-256-GCM, HMAC-SHA256, bcrypt

1.4.2 1.2 Frontend Applications

Component	Technology	Version	Purpose
Framework (Web)	React	19.x	UI library
Bundler	Vite	5.x	Build tool and dev server
Language	TypeScript	5.x	Type safety
Styling	Tailwind CSS	3.x	Utility-first CSS
Animations	Framer Motion	12.x	Animation library
Icons	Lucide React	—	Icon library
State (Web)	React Context	—	Global state management
State (Mobile)	Zustand	5.x	Global state management
Framework (Mobile)	React Native + Expo	0.86 / SDK 52	Mobile framework

1.4.3 1.3 Infrastructure

Component	Technology	Purpose
Containerization	Docker + Docker Compose	Local development
Orchestration	Kubernetes	Production deployment
Reverse Proxy	Nginx	Production static serving + routing
CI/CD	GitHub Actions / GitLab CI	Automated testing and deployment
Registry	GitHub Container Registry (ghcr.io)	Docker image storage
Load Testing	k6	Performance testing

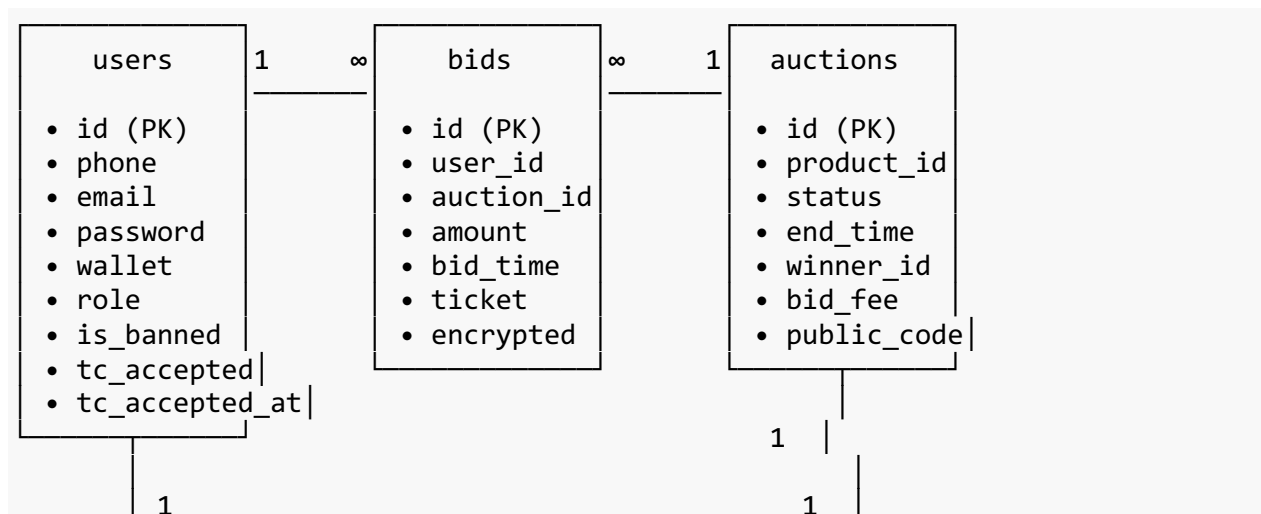
Component	Technology	Purpose
Caching	Redis 7.x	Application-level caching, response caching, rate limiting counters, session store

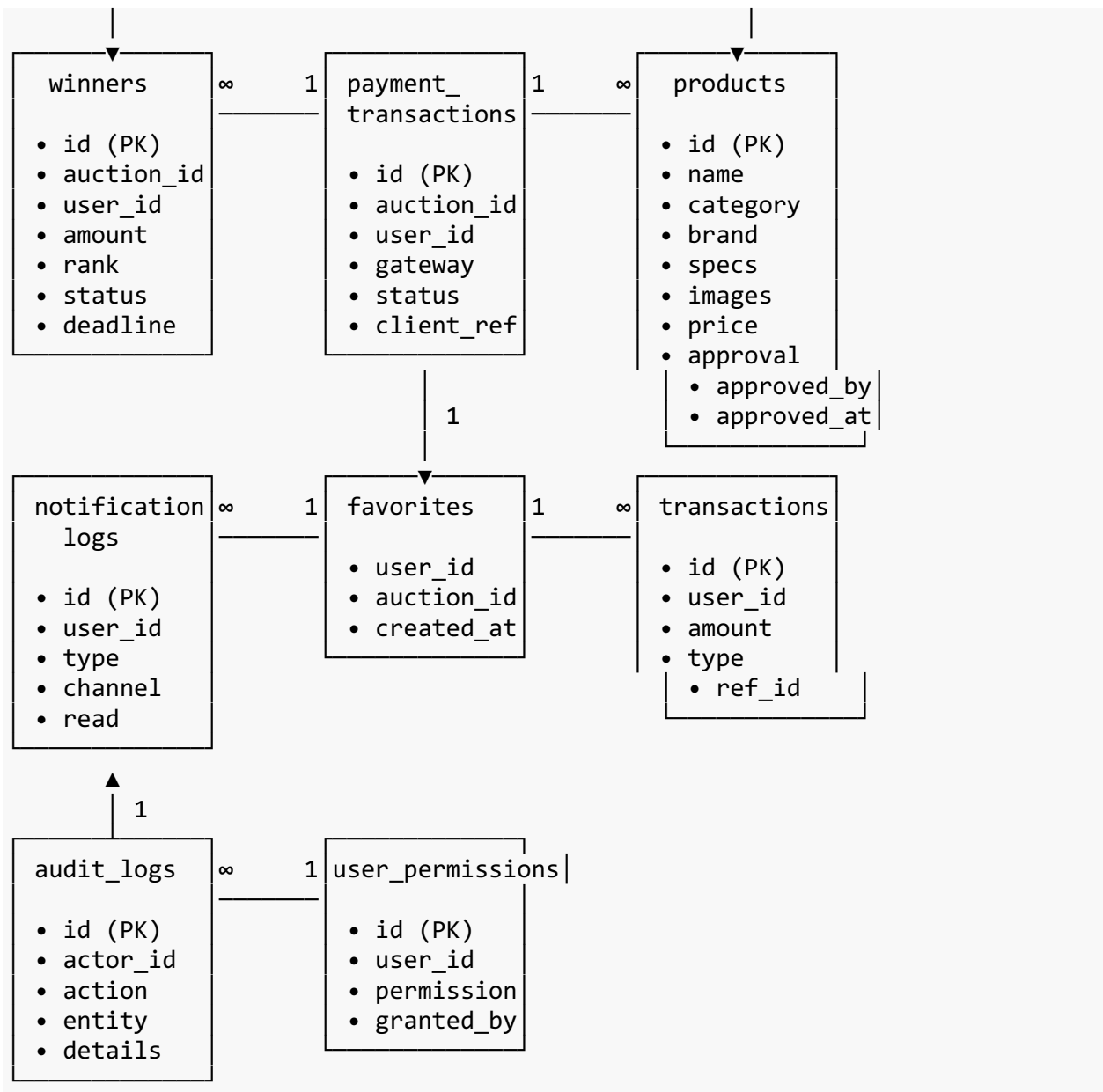
1.4.4 1.4 Caching Stack

Layer	Technology	Purpose	TTL Strategy
Application Cache	Redis 7.x	Frequently accessed read data (auction lists, product details, stats)	60s–300s depending on entity
Session Cache	Redis 7.x	Better-auth session tokens, JWT denylist	Sliding window, 7-day max
Rate Limit Cache	Redis 7.x	Throttle counters, bid nonce store	60s–900s
Query Result Cache	Redis 7.x + NestJS CacheInterceptor	Paginated query results, aggregated stats	30s–120s
WebSocket Adapter	Redis Pub/Sub	Cross-instance Socket.io message fan-out	N/A (event stream)

1.5 2. Database Schema (ERD)

1.5.1 2.1 Entity-Relationship Diagram





1.5.2 2.2 Table Definitions

1.5.2.12.2.1 users

Column	Type	Constraints	Index	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	PK	Unique user identifier
phone_number	VARCHAR(20)	UNIQUE, NOT NULL	idx_users_phone_number	Phone number (primary login)
email	VARCHAR	—	idx_users_email	Optional email address

Column	Type	Constraints	Index	Description
	R(255)		email	
password_hash	VARCHAR(255)	—	—	bcrypt hash (cost 12)
wallet_balance	DECIMAL(12,2)	NOT NULL DEFAULT 0	—	Current wallet balance in ETB
full_name	VARCHAR(255)	—	—	Display name
avatar_url	VARCHAR(255)	—	—	Profile image URL
hashed_refresh_token	VARCHAR(255)	—	—	bcrypt hash of refresh token
auth_provider	VARCHAR(20)	NOT NULL DEFAULT 'LOCAL'	—	LOCAL, TELEBIRR, BANKING_API, SUPER_APP
provider_id	VARCHAR(255)	—	—	External provider user ID
phone_verified	BOOLEAN	NOT NULL DEFAULT FALSE	—	Phone verification status
role	VARCHAR(20)	NOT NULL DEFAULT 'user'	idx_users_role	user or admin
is_banned	BOOLEAN	NOT NULL DEFAULT FALSE	—	Ban status
wallet_pin_hash	VARCHAR(60)	—	—	bcrypt hash (cost 10)
pin_attempts	INT	DEFAULT 0	—	Failed PIN attempt counter
pin_locked_until	TIMESTAMP	—	—	PIN lockout expiration
fcm_token	VARCHAR(255)	—	idx_users_fcm_token	Firebase Cloud Messaging token
apns_token	VARCHAR(255)	—	—	Apple Push Notification token
tc_accepted	BOOLEAN	NOT NULL DEFAULT FALSE	—	Terms and Conditions acceptance flag
tc_accepted_at	TIMESTAMP	—	idx_users_tc_accepted_at	Timestamp of most recent T&C acceptance
created_at	TIMESTAMP	NOT NULL DEFAULT	idx_users_	Account creation

Column	Type	Constraints	Index	Description
	MP	NOW()	created_at	timestamp

1.5.2.22.2.2 products

Column	Type	Constraints	Index	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	PK	Unique product identifier
name	VARCHAR(255)	NOT NULL	idx_products_name	Product display name
description	TEXT	—	—	Detailed product description
image_urls	JSONB	—	—	Array of image URLs
current_market_price	DECIMAL(12,2)	NOT NULL	—	Reference market price
brand	VARCHAR(255)	—	—	Product brand/manufacturer
specs	JSONB	—	—	Technical specifications
category	VARCHAR(80)	NOT NULL DEFAULT 'Electronics'	idx_products_category	Product category
approval_status	VARCHAR(20)	NOT NULL DEFAULT 'PENDING'	idx_products_approval_status	PENDING, APPROVED, REJECTED
approved_by	UUID	FK → users(id)	idx_products_approved_by	Admin who approved the product
approved_at	TIMESTAMP	—	—	Approval timestamp
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	—	Creation timestamp

1.5.2.32.2.3 auctions

Column	Type	Constraints	Index	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	PK	Unique auction identifier
product_id	UUID	FK → products(id) ON DELETE CASCADE	idx_auctions_product_id	Associated product

Column	Type	Constraints	Index	Description
start_time	TIMESTAMP	NOT NULL	—	Auction start time
end_time	TIMESTAMP	NOT NULL	idx_auctions_end_time	Auction end time
status	auction_status	NOT NULL DEFAULT 'ACTIVE'	idx_auctions	ACTIVE, CLOSED, EXPIRED
winner_user_id	UUID	FK → users(id)	idx_auctions_winner	Winning user (partial index)
winning_bid_amount	DECIMAL (12,2)	—	—	Winning bid amount
min_bid	DECIMAL (12,2)	—	—	Minimum bid threshold
max_bid	DECIMAL (12,2)	—	—	Maximum bid count
num_winners	INT	DEFAULT 1	—	Number of winners
payment_status	VARCHAR(20)	—	—	PENDING, PAID, EXPIRED
payment_deadline	TIMESTAMP	—	—	Winner payment deadline
last_payment_update	TIMESTAMP	—	—	Last payment status change
public_code	VARCHAR(5)	NOT NULL DEFAULT LPAD(nextval('auction_public_code_seq')::text, 5, '0')	idx_auctions_public_code UNIQUE	5-char public auction code
extensions	INTEGER	NOT NULL DEFAULT 0	—	Number of time extensions
bid_fee	DECIMAL (12,2)	NULL	—	Service fee per bid
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	—	Creation timestamp

1.5.2.42.2.4 bids (Primary Table)

Column	Type	Constraints	Index	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	PK	Unique bid identifier
user_id	UUID	FK → users(id) ON DELETE CASCADE	idx_bids_user_id	Bidding user

Column	Type	Constraints	Index	Description
auction_id	UUID	FK → auctions(id) ON DELETE CASCADE	idx_bids_auction_id	Target auction
amount	DECIMAL (12,2)	NOT NULL	idx_bids_auction_amount	Bid amount (ETB)
bid_time	TIMESTAMP	NOT NULL DEFAULT NOW()	idx_bids_bid_time	Bid submission time
service_fee_paid	BOOLEAN	NOT NULL DEFAULT TRUE	—	Whether bid fee was paid
ticket_number	VARCHAR	NOT NULL DEFAULT ''	idx_bids_ticket_number	Unique ticket (BID_ + hex)
encrypted_amount	TEXT	DEFAULT ''	—	AES-256-GCM encrypted amount

Constraints: - uq_bids_auction_user_time: UNIQUE (auction_id, user_id, bid_time)

1.5.2.52.2.5 bids_partitioned

Column	Type	Constraints	Description
id	UUID	NOT NULL	Partitioned copy of bids
user_id	UUID	FK → users(id) ON DELETE CASCADE	Distribution column
auction_id	UUID	FK → auctions(id) ON DELETE CASCADE	Target auction
amount	DECIMAL(12,2)	NOT NULL	Bid amount
bid_time	TIMESTAMP	NOT NULL DEFAULT NOW()	Bid submission time
service_fee_paid	BOOLEAN	NOT NULL DEFAULT TRUE	Fee payment status

Primary Key: (id, bid_time) **Partitioning:** PARTITION BY RANGE (bid_time) **Partitions:** bids_2026_q3 (2026-07-01 to 2026-10-01)

1.5.2.62.2.6 transactions

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	Transaction identifier
user_id	UUID	FK → users(id) ON DELETE CASCADE	Account holder

Column	Type	Constraints	Description
amount	DECIMAL(12,2)	NOT NULL	Transaction amount
type	transaction_type	NOT NULL	DEPOSIT, BID_FEE, REFUND
reference_id	VARCHAR(255)	—	External reference
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Transaction timestamp

1.5.2.72.2.7 favorites

Column	Type	Constraints	Description
user_id	UUID	FK → users(id) ON DELETE CASCADE, PK (composite)	User who favorited
auction_id	UUID	FK → auctions(id) ON DELETE CASCADE, PK (composite)	Favorited auction
created_at	TIMESTAMP	DEFAULT NOW()	When favorited

1.5.2.82.2.8 otps

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	OTP identifier
phone_number	VARCHAR(20)	NOT NULL, UNIQUE	Recipient phone
code	VARCHAR(6)	NOT NULL	OTP code
expires_at	TIMESTAMP	NOT NULL	Expiration time
verified	BOOLEAN	NOT NULL DEFAULT FALSE	Verification status
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Creation time

Citus: Registered as reference table (replicated to all nodes).

1.5.2.92.2.9 payment_transactions

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	Payment record ID
auction_id	UUID	NOT NULL	Associated auction

Column	Type	Constraints	Description
user_id	UUID	NOT NULL	Paying user
amount	DECIMAL(12,2)	NOT NULL	Payment amount
client_reference_id	VARCHAR(128)	NOT NULL, UNIQUE	Client-generated reference
sikina_payment_reference_id	VARCHAR(255)	—	SikinaPay reference
sikina_payment_url	VARCHAR	—	SikinaPay payment URL
awash_payment_url	VARCHAR	—	Awash payment URL
awash_transaction_id	VARCHAR(255)	—	Awash transaction ID
gateway	payment_gateway	NOT NULL DEFAULT 'SIKINAPAY'	SIKINAPAY or AWASH
customer_phone	VARCHAR(50)	—	Customer phone number
status	payment_transaction_status	NOT NULL DEFAULT 'PENDING'	PENDING, SUCCESSFUL, FAILED, etc.
currency	VARCHAR	—	Currency code
webhook_payload	JSONB	—	Raw webhook payload
payment_type	payment_type	NOT NULL DEFAULT 'WINNING_BID'	BID_FEE or WINNING_BID
retry_count	INT	NOT NULL DEFAULT 0	Reconciliation retries
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Creation time
updated_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Last update time
encrypted_amount	VARCHAR(512)	DEFAULT ''	Encrypted amount for closed auctions

1.5.2.10 2.2.10 winners

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	Winner record ID
auction_id	UUID	FK → auctions(id) ON DELETE CASCADE	Associated auction

Column	Type	Constraints	Description
user_id	UUID	FK → users(id) ON DELETE CASCADE	Winning user
amount	DECIMAL(12,2)	NOT NULL	Winning bid amount
rank	INTEGER	NOT NULL DEFAULT 1	Winner rank (1st, 2nd, etc.)
payment_status	VARCHAR(20)	DEFAULT 'PENDING'	PENDING, PAID, EXPIRED
payment_deadline	TIMESTAMP	—	Payment due time
notified_at	TIMESTAMP	—	When winner was notified
created_at	TIMESTAMP	DEFAULT NOW()	Record creation

Constraints: - uq_winner_auction_amount: UNIQUE (auction_id, amount) -
uq_winner_auction_rank: UNIQUE (auction_id, rank)

1.5.2.11 2.2.11 notification_logs

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	Notification ID
user_id	UUID	FK → users(id) ON DELETE CASCADE	Recipient
auction_id	UUID	FK → auctions(id) ON DELETE SET NULL	Related auction
type	VARCHAR(50)	NOT NULL	Notification type
channel	VARCHAR(20)	NOT NULL DEFAULT 'PUSH'	PUSH, SMS, INAPP
title	VARCHAR(255)	NOT NULL	Notification title
body	TEXT	NOT NULL	Notification body
metadata	JSONB	—	Additional data
read	BOOLEAN	DEFAULT FALSE	Read status
sent_at	TIMESTAMP	DEFAULT NOW()	Send timestamp
created_at	TIMESTAMP	DEFAULT NOW()	Creation timestamp

1.5.2.12 2.2.12 audit_logs

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	Log entry ID
actor_id	VARCHAR	NOT NULL	Actor identifier (user ID or system)
actor_phone	VARCHAR	—	Actor phone for context
action	VARCHAR	NOT NULL	Action performed
entity_type	VARCHAR	NOT NULL	Entity affected (user, auction, etc.)
entity_id	VARCHAR	NOT NULL	Entity identifier
details	JSONB	—	Action details
created_at	TIMESTAM P	NOT NULL DEFAULT NOW()	Log timestamp

1.5.2.13 2.2.13 user_permissions

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT uuid_generate_v4()	Permission ID
user_id	UUID	FK → users(id) ON DELETE CASCADE	User with permission
permission	VARCHAR(100)	NOT NULL	Permission string
granted_by	UUID	FK → users(id)	Admin who granted
created_at	TIMESTAM P	NOT NULL DEFAULT NOW()	Grant timestamp

Constraints: - uq_user_permission: UNIQUE (user_id, permission)

1.5.3 2.3 Prisma Schema (Excerpt)

The database schema is defined declaratively in `prisma/schema.prisma` and serves as the single source of truth. Prisma Client generates fully type-safe query builders at build time.

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

```

enum AuctionStatus {
    ACTIVE
    CLOSED
    EXPIRED
}

enum TransactionType {
    DEPOSIT
    BID_FEE
    REFUND
}

enum PaymentGateway {
    SIKINAPAY
    AWASH
}

enum ProductApprovalStatus {
    PENDING
    APPROVED
    REJECTED
}

model User {
    id                String      @id @default(uuid())
    phoneNumber       String      @unique @map("phone_number")
    email             String?     @map("email")
    passwordHash      String?     @map("password_hash")
    walletBalance     Decimal     @default(0) @map("wallet_balance") @db.Decimal(12, 2)
    fullName          String?     @map("full_name")
    avatarUrl         String?     @map("avatar_url")
    hashedRefreshToken String?     @map("hashed_refresh_token")
    authProvider      String      @default("LOCAL") @map("auth_provider")
    providerId        String?     @map("provider_id")
    phoneVerified     Boolean     @default(false) @map("phone_verified")
    role              String      @default("user")
    isBanned          Boolean     @default(false) @map("is_banned")
    walletPinHash     String?     @map("wallet_pin_hash")
    pinAttempts       Int         @default(0) @map("pin_attempts")
    pinLockedUntil    DateTime?   @map("pin_locked_until")
    fcmToken          String?     @map("fcm_token")
    apnsToken         String?     @map("apns_token")
    tcAccepted        Boolean     @default(false) @map("tc_accepted")
    tcAcceptedAt      DateTime?   @map("tc_accepted_at")
    createdAt         DateTime    @default(now()) @map("created_at")

    bids              Bid[]

```

```

    transactions      Transaction[]
    winners            Winner[]
    favorites           Favorite[]
    notifications       NotificationLog[]
    permissions         UserPermission[]
    approvedProducts    Product[]    @relation("ProductApprover")
    auditLogs           AuditLog[]

    @@index([role])
    @@index([email])
    @@index([fcmToken])
    @@index([createdAt])
    @@index([tcAcceptedAt])
    @@map("users")
}

model Product {
  id          String    @id @default(uuid())
  name        String
  description  String?
  imageUrls   Json?     @map("image_urls")
  currentMarketPrice Decimal @map("current_market_price") @db.Decimal(12,
2)
  brand       String?
  specs       Json?
  category    String    @default("Electronics")
  approvalStatus String @default("PENDING") @map("approval_status")
  approvedBy  String?   @map("approved_by")
  approvedAt  DateTime? @map("approved_at")
  createdAt   DateTime  @default(now()) @map("created_at")

  approver    User?      @relation("ProductApprover", fields: [appro
vedBy], references: [id])
  auctions    Auction[]

  @@index([name])
  @@index([category])
  @@index([approvalStatus])
  @@index([approvedBy])
  @@map("products")
}

model Auction {
  id          String    @id @default(uuid())
  productId   String    @map("product_id")
  startTime   DateTime  @map("start_time")
  endTime     DateTime  @map("end_time")
  status      AuctionStatus @default(ACTIVE)
  winnerUserId String?   @map("winner_user_id")

```

```

    winningBidAmount    Decimal?    @map("winning_bid_amount") @db.Decimal(12,
2)
    minBid              Decimal?    @map("min_bid") @db.Decimal(12, 2)
    maxBid              Decimal?    @map("max_bid") @db.Decimal(12, 2)
    numWinners          Int         @default(1) @map("num_winners")
    paymentStatus       String?     @map("payment_status")
    paymentDeadline     DateTime?   @map("payment_deadline")
    lastPaymentUpdate   DateTime?   @map("last_payment_update")
    publicCode          String      @map("public_code")
    extensions          Int         @default(0)
    bidFee              Decimal?    @map("bid_fee") @db.Decimal(12, 2)
    createdAt           DateTime    @default(now()) @map("created_at")

    product             Product     @relation(fields: [productId], references:
[id], onDelete: Cascade)
    bids                Bid[]
    winners              Winner[]

    @@index([productId])
    @@index([endTime])
    @@index([status])
    @@index([winnerUserId])
    @@index([publicCode], type: Hash)
    @@map("auctions")
}

model Bid {
    id                  String       @id @default(uuid())
    userId              String       @map("user_id")
    auctionId           String       @map("auction_id")
    amount              Decimal      @db.Decimal(12, 2)
    bidTime             DateTime     @default(now()) @map("bid_time")
    serviceFeePaid      Boolean      @default(true) @map("service_fee_paid")
    ticketNumber         String       @default("") @map("ticket_number")
    encryptedAmount     String       @default("") @map("encrypted_amount")

    user                User         @relation(fields: [userId], references: [i
d], onDelete: Cascade)
    auction             Auction      @relation(fields: [auctionId], references:
[id], onDelete: Cascade)

    @@unique([auctionId, userId, bidTime], name: "uq_bids_auction_user_time")
    @@index([userId])
    @@index([auctionId])
    @@index([bidTime])
    @@index([auctionId, amount])
    @@index([ticketNumber])
    @@map("bids")
}

```

```

model AuditLog {
  id                String    @id @default(uuid())
  actorId           String    @map("actor_id")
  actorPhone        String?   @map("actor_phone")
  action            String
  entityType        String    @map("entity_type")
  entityId          String    @map("entity_id")
  details           Json?
  createdAt         DateTime  @default(now()) @map("created_at")

  @@index([actorId])
  @@index([action])
  @@index([createdAt])
  @@map("audit_logs")
}

```

1.5.4 2.4 Database Index Catalog

Index Name	Table	Columns	Type	Partial	Description
idx_users_phone_number	users	phone_number	B-tree	No	Fast user lookup by phone
idx_users_email	users	email	B-tree	No	Fast user lookup by email
idx_users_role	users	role	B-tree	No	Fast admin/user filtering
idx_users_created_at	users	created_at	B-tree	No	User registration analytics
idx_users_fcm_token	users	fcm_token	B-tree	No	Push notification targeting
idx_users_tc_accepted_at	users	tc_accepted_at	B-tree	No	T&C acceptance reporting
idx_products_name	products	name	B-tree	No	Product search
idx_products_category	products	category	B-tree	No	Category filtering
idx_products_approval_status	products	approval_status	B-tree	No	Approval workflow filtering
idx_products_approved_by	products	approved_by	B-tree	No	Approver audit trail
idx_auctions_status	auctions	status	B-tree	No	Status-based filtering
idx_auctions_end_time	auctions	end_time	B-tree	No	Expiry cron queries

Index Name	Table	Columns	Type	Partial	Description
idx_auctions_product_id	auctions	product_id	B-tree	No	Product-auction joins
idx_auctions_winner	auctions	winner_user_id	B-tree	WHERE winner_user_id IS NOT NULL	Winner lookups
idx_auctions_active_lookup	auctions	(end_time, id)	B-tree	WHERE status = 'ACTIVE'	Active auction queries
idx_auctions_public_code	auctions	public_code	B-tree	UNIQUE	Public code lookup
idx_bids_user_id	bids	user_id	B-tree	No	User bid history
idx_bids_auction_id	bids	auction_id	B-tree	No	Auction bid history
idx_bids_bid_time	bids	bid_time	B-tree	No	Time-based queries
idx_bids_auction_amount	bids	(auction_id, amount)	B-tree	No	LUB frequency lookups
idx_bids_auction_lookup	bids	(auction_id, bid_time, amount)	B-tree	No	Auction detail queries
idx_bids_user_auction	bids	(user_id, auction_id, amount)	B-tree	No	Duplicate detection
idx_bids_ticket_number	bids	ticket_number	B-tree	No	Ticket lookups
uq_bids_auction_user_time	bids	(auction_id, user_id, bid_time)	UNIQUE	No	Prevent duplicate bids
idx_transactions_user_id	transactions	user_id	B-tree	No	User transaction history
idx_transactions_reference_id	transactions	reference_id	B-tree	No	External reference lookup
idx_transactions_created_at	transactions	created_at	B-tree	No	Time-based reporting
idx_favorites_user_id	favorites	user_id	B-tree	No	User favorites
idx_favorites_auction_id	favorites	auction_id	B-tree	No	Auction favorites

Index Name	Table	Columns	Type	Partial	Description
					count
idx_favorites_created_at	favorites	created_at	B-tree	No	Recent favorites
idx_otps_phone_number	otps	phone_number	B-tree	UNIQUE	OTP lookup
idx_otps_expires_at	otps	expires_at	B-tree	No	Cleanup expired OTPs
idx_payment_transactions_auction_id	payment_transactions	auction_id	B-tree	No	Auction payment history
idx_payment_transactions_user_id	payment_transactions	user_id	B-tree	No	User payment history
idx_payment_transactions_client_reference_id	payment_transactions	client_reference_id	B-tree	UNIQUE	Webhook matching
idx_payment_transactions_status	payment_transactions	status	B-tree	No	Reconciliation queries
idx_payment_transactions_gateway	payment_transactions	gateway	B-tree	No	Gateway filtering
idx_payment_transactions_awash_transaction_id	payment_transactions	awash_transaction_id	B-tree	No	Awash lookup
idx_winners_auction_id	winners	auction_id	B-tree	No	Auction winners
idx_winners_user_id	winners	user_id	B-tree	No	User wins
idx_winners_payment_status	winners	payment_status	B-tree	No	Payment tracking
uq_winner_auction_amount	winners	(auction_id, amount)	UNIQUE	No	Prevent duplicate winners
uq_winner_auction_rank	winners	(auction_id, rank)	UNIQUE	No	Prevent duplicate ranks
idx_notification_logs_user_id	notification_logs	user_id	B-tree	No	User notifications
idx_notification_logs_type	notification_logs	type	B-tree	No	Type filtering

Index Name	Table	Columns	Type	Partial	Description
idx_notification_logs_read	notification_logs	(user_id, read)	B-tree	No	Unread count
idx_audit_logs_actor_id	audit_logs	actor_id	B-tree	No	Actor audit trail
idx_audit_logs_action	audit_logs	action	B-tree	No	Action filtering
idx_audit_logs_created_at	audit_logs	created_at	B-tree	No	Time-range queries
idx_user_permissions_user_id	user_permissions	user_id	B-tree	No	Permission lookups
uq_user_permission	user_permissions	(user_id, permission)	UNIQUE	No	Prevent duplicate permissions

1.5.5 2.5 Database Functions and Sequences

```
-- Sequence for 5-char public auction codes
CREATE SEQUENCE auction_public_code_seq START 1;

-- Custom types
CREATE TYPE auction_status AS ENUM ('ACTIVE', 'CLOSED', 'EXPIRED');
CREATE TYPE transaction_type AS ENUM ('DEPOSIT', 'BID_FEE', 'REFUND');
CREATE TYPE payment_transaction_status AS ENUM ('PENDING', 'SUCCESSFUL', 'FAILED', 'EXPIRED', 'CANCELLED', 'REVOKED');
CREATE TYPE payment_type AS ENUM ('BID_FEE', 'WINNING_BID', 'WALLET');
CREATE TYPE payment_gateway AS ENUM ('SIKINAPAY', 'AWASH');
CREATE TYPE product_approval_status AS ENUM ('PENDING', 'APPROVED', 'REJECTED');
```

1.6 3. API Contracts

1.6.1 3.1 API Overview

Service	Base URL	Port	Protocol	Auth
Identity Service	http://localhost:3001/api/v1	3001	HTTP REST	Better-auth JwtAuthGuard / Internal API Key
Auction Engine	http://localhost:3002/api/v1	3002	HTTP REST + WebSocket	Better-auth JwtAuthGuard

Service	Base URL	Port	Protocol	Auth
Query Service	http://localhost:3003/api/v1	3003	HTTP REST	ard / Internal API Key Better-auth JwtAuthGuard / Cache
WebSocket	ws://localhost:3002/auctions	3002	Socket.io	Better-auth JwtAuthGuard

Total Endpoints: 120+ HTTP endpoints + 1 WebSocket gateway

Authentication: All authenticated endpoints use Better-auth with the JwtAuthGuard NestJS guard. Better-auth provides session management, token issuance, refresh-token rotation, and pluggable OAuth strategies (TeleBirr, Banking API, Super-App). The JwtAuthGuard validates the bearer access token on every request, checks the Redis-backed session store for revocation, and populates the request user context. JWT secret is shared identically across all three services so that tokens issued by identity-service are verifiable everywhere.

Cache Headers: All cacheable responses from Query Service and read-heavy endpoints include Redis cache headers:

```
X-Cache-Status: HIT | MISS | BYPASS | STALE
X-Cache-Key: auctions:active:page1:limit20
X-Cache-TTL: 120
Cache-Control: public, max-age=120, stale-while-revalidate=60
ETag: "33a64df551425fcc55e4d42a1447fc24"
```

1.6.2 3.2 Identity Service API

Base URL: http://localhost:3001/api/v1

1.6.2.13.2.1 Authentication

Method	Path	Auth	Rate Limit	Description
POST	/auth/register	None	—	Register new user
POST	/auth/login/phone	None	5/min per phone	Login with phone + password
POST	/auth/login/email	None	5/min per email	Login with email +

Method	Path	Auth	Rate Limit	Description
	1			password
POST	/auth/login/telebirr	None	—	OAuth login via TeleBirr
POST	/auth/login/banking	None	—	OAuth login via Banking API
POST	/auth/login/super-app/:provider	None	—	Login via super app
POST	/auth/refresh	None	—	Refresh JWT token
GET	/auth/profile	JwtAuthGuard	—	Get current user profile
PATCH	/auth/profile	JwtAuthGuard	—	Update current user profile
POST	/auth/fcm-token	JwtAuthGuard	—	Register FCM push token
POST	/auth/logout	JwtAuthGuard	—	Logout (invalidate refresh token)
POST	/otp/send	None	3/min per phone	Send OTP to phone
POST	/otp/verify	None	—	Verify OTP code
POST	/auth/terms/accept	JwtAuthGuard	—	Accept Terms and Conditions
GET	/auth/terms/status	JwtAuthGuard	—	Get T&C acceptance status

POST /auth/register

Request:

```
{
  "phone_number": "0911111111",
  "full_name": "Admin User",
  "password": "securePassword123"
}
```

Response 201:

```
{
  "id": "uuid",
```

```
"phone_number": "0911111111",
"full_name": "Admin User",
"role": "user",
"tc_accepted": false,
"access_token": "eyJhbG...",
"refresh_token": "eyJhbG..."
}
```

POST /auth/login/phone

Request:

```
{
  "phone_number": "0911111111",
  "password": "securePassword123"
}
```

Response 200:

```
{
  "access_token": "eyJhbG...",
  "refresh_token": "eyJhbG...",
  "user": { "id": "uuid", "phone_number": "...", "role": "user", "tc_accepted": true }
}
```

Error 401:

```
{
  "statusCode": 401,
  "errorCode": "ERR_AUTH_INVALID_CREDENTIALS",
  "message": "Invalid credentials"
}
```

POST /auth/refresh

Request:

```
{
  "refresh_token": "eyJhbG..."
}
```

Response 200:

```
{
  "access_token": "eyJhbG...",
  "refresh_token": "eyJhbG..."
}
```

POST /auth/tc/accept

Headers: Authorization: Bearer <access_token>

Response 200:

```
{
  "tc_accepted": true,
}
```

```
"tc_accepted_at": "2026-08-18T08:00:00Z"
}
```

GET /auth/tc/status

Headers: Authorization: Bearer <access_token>

Response 200:

```
{
  "tc_accepted": true,
  "tc_accepted_at": "2026-08-18T08:00:00Z",
  "tc_version": "2026-08-01"
}
```

1.6.2.23.2.2 Wallet

Method	Path	Auth	Description
GET	/wallet/balance	JwtAuthGuard	Get wallet balance
GET	/wallet/transactions	JwtAuthGuard	Get transaction history (paginated)
POST	/wallet/deposit	JwtAuthGuard	Initiate wallet deposit
POST	/wallet/webhook/fintech	Webhook sig	Handle fintech payment webhook
POST	/wallet/set-pin	JwtAuthGuard	Set wallet PIN
POST	/wallet/verify-pin	JwtAuthGuard	Verify wallet PIN
GET	/wallet/has-pin	JwtAuthGuard	Check if PIN is set
GET	/wallet/pin-status	JwtAuthGuard	Get PIN setup status
POST	/wallet/deduct-fee	JwtAuthGuard or Internal	Deduct bid fee from wallet
GET	/wallet/user/:id	JwtAuthGuard	Resolve user name by ID
GET	/wallet/user/:id/internal	Internal	Resolve user name by ID (internal)

GET /wallet/balance

Headers: Authorization: Bearer <access_token>

Response 200:

```
{
  "balance": 5000.00,
  "currency": "ETB"
}
```

```
}  
X-Cache-Status: BYPASS
```

POST /wallet/deduct-fee

Headers: Authorization: Bearer <access_token> OR x-internal-api-key: <key>

Request:

```
{  
  "amount": 5.00,  
  "auction_id": "uuid",  
  "user_id": "uuid"  
}
```

Response 200:

```
{  
  "new_balance": 4995.00,  
  "transaction_id": "uuid"  
}
```

1.6.2.33.2.3 Admin Users

Method	Path	Auth	Permission	Description
GET	/admin/users	JwtAuthGuard + admin	users:read	List users (paginated)
GET	/admin/users/export/csv	JwtAuthGuard + admin	export	Export users as CSV
GET	/admin/users/:id	JwtAuthGuard + admin	users:read	Get single user
GET	/admin/users/:id/detail	JwtAuthGuard + admin	users:read	Get user detail
GET	/admin/users/:id/transactions	JwtAuthGuard + admin	transactions:read	Get user transactions
PATCH	/admin/users/:id/role	JwtAuthGuard + admin	users:role	Update user role
PATCH	/admin/users/:id/ban	JwtAuthGuard + admin	users:ban	Toggle user ban
POST	/admin/users/bulk	JwtAuthGuard +	users:role	Bulk update roles

Method	Path	Auth	Permission	Description
POST	k/role /admin/users/bulk/ban	admin JwtAuthGuard + admin	users:ban	Bulk toggle bans
GET	/admin/users/transactions/all	JwtAuthGuard + admin	transactions:read	List all transactions
GET	/admin/users/transactions/export/csv	JwtAuthGuard + admin	export	Export transactions
GET	/admin/users/audit/list	JwtAuthGuard + admin	audit:read	List audit logs
GET	/admin/users/permissions/list	JwtAuthGuard + admin	users:permissions	List all permissions
GET	/admin/users/:id/permissions	JwtAuthGuard + admin	users:permissions	Get user permissions
POST	/admin/users/:id/permissions/grant	JwtAuthGuard + admin	users:permissions	Grant permissions
POST	/admin/users/:id/permissions/revoke	JwtAuthGuard + admin	users:permissions	Revoke permissions
POST	/admin/audit/log	Internal	—	Log audit event

GET /admin/users

Headers: Authorization: Bearer <access_token>

Query: ?page=1&limit=20&search=0911&role=user

Response 200:

```
{
  "data": [...],
  "total": 30,
```

```
"page": 1,
"limit": 20
}
X-Cache-Status: MISS
X-Cache-Key: admin:users:page1:limit20:search0911
```

PATCH /admin/users/:id/ban

Headers: Authorization: Bearer <access_token>

Request:

```
{
  "is_banned": true
}
```

Response 200:

```
{
  "id": "uuid",
  "is_banned": true
}
```

1.6.2.43.2.4 Notifications

Method	Path	Auth	Description
POST	/notify/winner	Internal	Send winner notification
POST	/notify/winner-bulk	Internal	Bulk send winner notifications
POST	/notify/bid-confirmation	Internal	Send bid confirmation
POST	/notify/outbid	Internal	Send outbid alert
POST	/notify/auction-started	Internal	Send auction started notification
POST	/notify/auction-extended	Internal	Send auction extended notification
POST	/notify/max-bid-reached	Internal	Send max bid reached notification
POST	/notify/auction-fair-play-extended	Internal	Send fair play extended notification
POST	/notify/auction-forced-closure	Internal	Send forced closure notification
POST	/notify/ending-soon	Internal	Send auction ending soon notification
GET	/notify/inbox	JwtAuthGuard	Get in-app notifications

Method	Path	Auth	Description
	x		
POST	/notify/inbox/:id/read	JwtAuthGuard	Mark notification as read
POST	/notify/inbox/read-all	JwtAuthGuard	Mark all notifications as read

1.6.2.53.2.5 Health

Method	Path	Auth	Description
GET	/health	None	Health check (DB + Redis, 503 if degraded)

1.6.3 3.3 Auction Engine API

Base URL: http://localhost:3002/api/v1

1.6.3.13.3.1 Bidding

Method	Path	Auth	Guards	Description
POST	/auctions/:id/bid	JwtAuthGuard	ThrottleGuard, NonceGuard, BiddingWindowInterceptor	Place bid (202 Accepted)
GET	/auctions/:id/my-bids	JwtAuthGuard	—	Get current user's bids
GET	/auctions/:id/result	JwtAuthGuard	—	Get auction result with winner

POST /auctions/:id/bid

Headers:

```
Authorization: Bearer <access_token>
x-bid-nonce: <unique-nonce>
x-bid-timestamp: 1693123456789
```

Request:

```
{
  "amount": 15.50
}
```

Response 202 Accepted:

```
{
  "ticket_number": "BID_a1b2c3d4e5f6",
  "auction_id": "uuid",
  "amount": 15.50,
  "message": "Bid placed successfully"
}
```

```

}

Error 429:
{
  "statusCode": 429,
  "errorCode": "ERR_BID_RATE_LIMIT",
  "message": "Too many bids. Please slow down."
}

```

GET /auctions/:id/result

Headers: Authorization: Bearer <access_token>

Response 200:

```

{
  "auction_id": "uuid",
  "status": "CLOSED",
  "winner": {
    "user_id": "uuid",
    "full_name": "Winner Name",
    "amount": 12.00
  },
  "total_bids": 150,
  "unique_bidders": 45
}
X-Cache-Status: HIT
X-Cache-Key: auction:result:uuid
X-Cache-TTL: 300

```

1.6.3.23.3.2 Admin Auctions and Products

Method	Path	Auth	Description
GET	/admin/products	JwtAuthGuard + admin	List products (paginated)
GET	/admin/products/export/csv	JwtAuthGuard + admin	Export products as CSV
POST	/admin/products	JwtAuthGuard + admin	Create product
PATCH	/admin/products/:id	JwtAuthGuard + admin	Update product
POST	/admin/products/:id/download-images	JwtAuthGuard + admin	Download product images
POST	/admin/products/download-all-images	JwtAuthGuard + admin	Download all product images
DELETE	/admin/products/:id	JwtAuthGuard + admin	Delete product

Method	Path	Auth	Description
POST	/admin/products/bulk-delete	JwtAuthGuard + admin	Bulk delete products
POST	/admin/products/:id/approve	JwtAuthGuard + admin	Approve product for auction use
POST	/admin/products/:id/reject	JwtAuthGuard + admin	Reject product (with reason)
GET	/admin/products/pending	JwtAuthGuard + admin	List products pending approval
GET	/admin/auctions	JwtAuthGuard + admin	List auctions (paginated)
GET	/admin/auctions/export/csv	JwtAuthGuard + admin	Export auctions as CSV
POST	/admin/auctions	JwtAuthGuard + admin	Create auction
PATCH	/admin/auctions/:id	JwtAuthGuard + admin	Update auction
DELETE	/admin/auctions/:id	JwtAuthGuard + admin	Delete auction
POST	/admin/auctions/bulk-delete	JwtAuthGuard + admin	Bulk delete auctions
POST	/admin/auctions/:id/close	JwtAuthGuard + admin	Close auction early
POST	/admin/auctions/:id/force-close	JwtAuthGuard + admin	Force close auction
GET	/admin/auctions/:id/winner	JwtAuthGuard + admin	Draw winner for auction
GET	/admin/auctions/:id/bids	JwtAuthGuard + admin	Get all bids for auction

POST /admin/products/:id/approve

Headers: Authorization: Bearer <access_token>

Response 200:

```
{
  "id": "uuid",
```

```

    "approval_status": "APPROVED",
    "approved_by": "admin-uuid",
    "approved_at": "2026-08-18T08:00:00Z"
  }

```

POST /admin/products/:id/reject

Headers: Authorization: Bearer <access_token>

>Request:

```

{
  "reason": "EVICTION: Product images do not meet quality standards"
}

```

Response 200:

```

{
  "id": "uuid",
  "approval_status": "REJECTED",
  "rejection_reason": "Product images do not meet quality standards"
}

```

1.6.3.33.3 Payments

Method	Path	Auth	Description
POST	/payments/:<unknown>	JwtAuthGuard	Create payment link for winner
POST	/payments/:auctionId/confirm	JwtAuthGuard	Confirm winning payment
GET	8:00:00Z"		
	}		

****GET /payments/:auctionId/status****

Headers: Authorization: Bearer Response 200: { "status": "PENDING", "gateway": "SIKINAPAY", "amount": 5000.00, "created_at": "2026-08-18T08:00:00Z" }

3.3.4 Settlement Reports

Method	Path	Auth	Description
GET	/admin/settlements`	JwtAuthGuard + admin	List settlement reports (paginated)
GET	/admin/settlements/:id`	JwtAuthGuard + admin	Get settlement report detail
POST	/admin/settlements/generate`	JwtAuthGuard + admin	Generate settlement report for date range
GET	/admin/settlements/:id/export/csv`	JwtAuthGuard + admin	Export s

```
ettlement report as CSV |  
| GET | `/admin/settlements/:id/export/pdf` | JwtAuthGuard + admin | Export s  
ettlement report as PDF |
```

****POST /admin/settlements/generate****

``json

Headers: Authorization: Bearer <access_token>

Request:

```
{  
  "start_date": "2026-08-01",  
  "end_date": "2026-08-31",  
  "gateway": "SIKINAPAY"  
}
```

Response 201:

```
{  
  "id": "uuid",  
  "status": "GENERATED",  
  "total_transactions": 145,  
  "total_amount": 725000.00,  
  "currency": "ETB",  
  "generated_at": "2026-08-18T08:00:00Z"  
}
```

1.6.3.43.3.5 Winner Management

Method	Path	Auth	Description
GET	/admin/winners	JwtAuthGuard + admin	List all winners (paginated, filterable)
GET	/admin/winners/:id	JwtAuthGuard + admin	Get winner detail
PATCH	/admin/winners/:id/payment-deadline	JwtAuthGuard + admin	Extend winner payment deadline
POST	/admin/winners/:id/notification	JwtAuthGuard + admin	Re-send winner notification
GET	/admin/winners/export/csv	JwtAuthGuard + admin	Export winners as CSV
GET	/admin/auctions/:id/winners	JwtAuthGuard + admin	Get all winners for an auction

PATCH /admin/winners/:id/payment-deadline

Headers: Authorization: Bearer <access_token>

Request:

```
{
  "payment_deadline": "2026-08-25T00:00:00Z"
}
```

Response 200:

```
{
  "id": "uuid",
  "payment_deadline": "2026-08-25T00:00:00Z",
  "payment_status": "PENDING"
}
```

1.6.3.53.3.6 Webhooks

Method	Path	Auth	Description
POST	/payments/webhook/awash	Webhook sig	Handle Awash payment events
POST	/payments/webhook/sikina	Webhook sig	Handle SikinaPay payment events

POST /payments/webhook/sikina

Headers:

```
sikinapay-signature: <hmac-sha256-signature>
```

Body (raw):

```
{
  "event": "payment.success",
  "data": {
    "client_reference_id": "uuid",
    "amount": 5000.00,
    "status": "SUCCESSFUL",
    "transaction_id": "gateway-tx-id"
  }
}
```

Response 200: { "received": true }

1.6.3.63.3.7 Health

Method	Path	Auth	Description
GET	/health	None	Health check (DB + Redis, 503 if degraded)
GET	/health/ready	None	Readiness check (DB only)

1.6.3.73.3.8 WebSocket Gateway

Namespace: /auctions

Event	Direction	Auth	Description
subscribe:auction	Client → Server	JwtAuthGuard	Subscribe to auction updates

Event	Direction	Auth	Description
unsubscribe:au ction	Client → Server	JwtAuthGua rd	Unsubscribe from auction
auction:update	Server → Client	—	Broadcast bid count update

```
// Client → Server
{
  "event": "subscribe:au",
  "data": { "auctionId": "uuid" }
}

// Server → Client
{
  "event": "auction:update",
  "data": {
    "auction_id": "uuid",
    "total_bids": 43,
    "timestamp": "2026-08-18T08:30:00Z"
  }
}
```

1.6.4 3.4 Query Service API

Base URL: http://localhost:3003/api/v1

1.6.4.13.4.1 Auctions

Method	Path	Auth	Cache	Description
GET	/auctions/ active	None	Yes (120s)	List active auctions
GET	/auctions/ closed	None	Yes (300s)	List closed auctions
GET	/auctions/ my-bids	JwtAuthGu ard	Yes (30s)	Get user's bid history
GET	/auctions/ my-wins	JwtAuthGu ard	Yes (60s)	Get user's won auctions
GET	/auctions/ :id	JwtAuthGu ard	Yes (120s)	Get auction by UUID
GET	/auctions/ :id/bids	JwtAuthGu ard	Yes (60s)	Get bid history for auction

GET /auctions/active

Response 200:

```
{
  "data": [
```

```
{
  "id": "uuid",
  "public_code": "00001",
  "product": { "name": "iPhone 15", "images": [...] },
  "start_time": "2026-08-15T00:00:00Z",
  "end_time": "2026-08-18T00:00:00Z",
  "status": "ACTIVE",
  "bid_fee": 5.00,
  "total_bids": 42,
  "unique_bidders": 18
},
"total": 24
}
X-Cache-Status: HIT
X-Cache-Key: auctions:active:page1:limit20
X-Cache-TTL: 120
Cache-Control: public, max-age=120, stale-while-revalidate=60
```

GET /auctions/:id

Response 200:

```
{
  "id": "uuid",
  "public_code": "00001",
  "product": {
    "id": "uuid",
    "name": "iPhone 15 Pro Max",
    "description": "...",
    "image_urls": ["https://..."],
    "current_market_price": 150000.00,
    "category": "Smartphones",
    "brand": "Apple",
    "specs": { "storage": "256GB", "chip": "A17 Pro" },
    "approval_status": "APPROVED"
  },
  "start_time": "2026-08-15T00:00:00Z",
  "end_time": "2026-08-18T00:00:00Z",
  "status": "ACTIVE",
  "bid_fee": 5.00,
  "min_bid": 1.00,
  "max_bid": 100,
  "extensions": 1,
  "total_bids": 42,
  "unique_bidders": 18
}
X-Cache-Status: HIT
X-Cache-Key: auction:detail:uuid
X-Cache-TTL: 120
```

1.6.4.23.4.2 Products

Method	Path	Auth	Cache	Description
GET	/products	None	Yes (300s)	List all products
GET	/products/:id	None	Yes (300s)	Get product by UUID

1.6.4.33.4.3 Favorites

Method	Path	Auth	Description
GET	/favorites	JwtAuthGuard	Get user's favorites
POST	/favorites/: auctionId	JwtAuthGuard	Add auction to favorites
DELETE	/favorites/: auctionId	JwtAuthGuard	Remove auction from favorites

GET /favorites

Headers: Authorization: Bearer <access_token>

Response 200:

```
{
  "data": [
    {
      "auction_id": "uuid",
      "product": { "name": "iPhone 15", "images": [...] },
      "end_time": "2026-08-18T00:00:00Z",
      "status": "ACTIVE"
    }
  ]
}
```

POST /favorites/:auctionId

Headers: Authorization: Bearer <access_token>

Response 201:

```
{
  "auction_id": "uuid",
  "created_at": "2026-08-18T08:00:00Z"
}
```

1.6.4.43.4.4 Admin

Method	Path	Auth	Description
GET	/admin/stats	JwtAuthGuard + admin	Get admin dashboard stats

GET /admin/stats

Headers: Authorization: Bearer <access_token>

Response 200:

```
{
  "users": { "total": 1000, "active": 850 },
}
```

```

    "auctions": { "active": 24, "closed": 156, "expired": 12 },
    "bids": { "total": 45230, "today": 1205 },
    "revenue": { "total": 125000.00, "today": 3400.00 },
    "topBidders": [...],
    "bidTrend": { "labels": [...], "data": [...] }
  }
X-Cache-Status: HIT
X-Cache-Key: admin:stats:dashboard
X-Cache-TTL: 60

```

1.6.4.53.4.5 Audit Log Viewer

Method	Path	Auth	Permission	Description
GET	/admin/audit/logs	JwtAuthGuard + admin	audit:read	List audit logs (paginated, filterable)
GET	/admin/audit/logs/:id	JwtAuthGuard + admin	audit:read	Get single audit log detail
GET	/admin/audit/logs/export/csv	JwtAuthGuard + admin	audit:read	Export audit logs as CSV
GET	/admin/actors/:actorId	JwtAuthGuard + admin	audit:read	Get all actions by an actor

GET /admin/audit/logs

```

Headers: Authorization: Bearer <access_token>
Query: ?page=1&limit=50&actor_id=uuid&action=place_bid&entity_type=auction&from=2026-08-01&to=2026-08-31
Response 200:
{
  "data": [
    {
      "id": "uuid",
      "actor_id": "uuid",
      "actor_phone": "0911111111",
      "action": "place_bid",
      "entity_type": "auction",
      "entity_id": "uuid",
      "details": { "amount": 15.50, "ticket_number": "BID_..." },
      "created_at": "2026-08-18T08:00:00Z"
    }
  ],
  "total": 5000,
  "page": 1,

```

```
"limit": 50
}
```

1.6.4.63.4.6 Analytics

Method	Path	Auth	Description
GET	/admin/analytics/overview	JwtAuthGuard + admin	Platform-wide analytics overview
GET	/admin/analytics/bids	JwtAuthGuard + admin	Bid analytics (volume, trends, distribution)
GET	/admin/analytics/revenue	JwtAuthGuard + admin	Revenue analytics (fees, payments, breakdown)
GET	/admin/analytics/users	JwtAuthGuard + admin	User analytics (registration, retention, activity)
GET	/admin/analytics/auctions	JwtAuthGuard + admin	Auction analytics (closure rate, duration, participation)
GET	/admin/analytics/export	JwtAuthGuard + admin	Export analytics report (CSV/PDF)

GET /admin/analytics/overview

Headers: Authorization: Bearer <access_token>

Query: ?period=30d

Response 200:

```
{
  "period": "30d",
  "total_users": 12500,
  "new_users": 1500,
  "total_auctions": 200,
  "total_bids": 45230,
  "total_revenue": 125000.00,
  "avg_bids_per_auction": 226,
  "avg_auction_duration_hours": 72,
  "conversion_rate": 0.085
}
```

X-Cache-Status: HIT

X-Cache-Key: analytics:overview:30d

X-Cache-TTL: 300

1.6.4.73.4.7 Health

Method	Path	Auth	Description
GET	/health	None	Health check (DB only, 503 if disconnected)

1.6.5 3.5 Proxy Configurations

1.6.5.13.5.1 Vite Dev Proxy (*takeLow-web/vite.config.ts*)

Frontend Path	Target Service	Notes
/api/v1/auth	identity-service	
/api/v1/wallet	identity-service	
/api/v1/notify	identity-service	
/api/v1/admin/users	identity-service	
/api/v1/payments	auction-engine	
/api/v1/admin/auctions	auction-engine	
/api/v1/admin/products	auction-engine	
/api/v1/admin/stats	query-service	
/api/v1/admin	auction-engine	
/api/v1/products	query-service	
/api/v1/auctions/result	auction-engine	
/api/v1/auctions/:id/result	auction-engine	Regex
/api/v1/auctions/:id/bid	auction-engine	Regex
/api/v1/auctions/:id/my-bids	auction-engine	Regex
/api/v1/auctions	query-service	
/api	query-service	Catch-all
/socket.io	auction-engine	WebSocket
/uploads	auction-engine	Static files

1.6.5.23.5.2 Nginx Production Proxy (*takeLow-web/nginx.conf*)

Path Pattern	Target
/api/v1/auth	http://identity-service:3000
/api/v1/wallet	http://identity-service:3000
/api/v1/notify	http://identity-service:3000
/api/v1/admin/users	http://identity-service:3000
/api/v1/auctions/:id/(bid\ result\ my-bids)	http://auction-engine:3000
/api/v1/payments	http://auction-engine:3000
/api/v1/admin/stats	http://query-service:3000
/api/v1/admin	http://auction-engine:3000
/api/v1/auctions	http://query-service:3000
/api/v1/products	http://query-service:3000

Path Pattern	Target
/socket.io	http://auction-engine:3000 (WebSocket upgrade)
/	Static file serving

1.6.5.33.5.3 DevProxy (scripts/dev-proxy.js)

Path	Target	Notes
/socket.io	auction-engine	WebSocket proxy
/api/v1/auth	identity-service	
/api/v1/wallet	identity-service	
/api/v1/admin/auction	auction-engine	
/api/v1/admin/product	auction-engine	
/api/v1/admin	query-service	
/api/v1/auctions/:id/bid	auction-engine	POST only
/api/v1/payments/webhook	auction-engine	
/api/v1/payments	auction-engine	
/api/v1/auctions	query-service	
/api/v1/products	query-service	
/api/v1/favorites	query-service	
/payment/success	Web app	302 redirect
/payment/failed	Web app	302 redirect
/api	query-service	Catch-all

1.7 4. Data Transfer Objects (DTOs)

1.7.1 4.1 Identity Service DTOs

CreateUserDto

```
class CreateUserDto {
    @IsString()
    @Length(10, 20)
    @Matches(/^\\+?[0-9]{10,15}$/)
    phone_number: string;

    @IsString()
    @Length(2, 255)
    full_name: string;

    @IsString()
    @Length(8, 255)
```

```
password: string;

@IsOptional()
@IsEmail()
email?: string;
}
```

LoginDto

```
class LoginDto {
  @IsString()
  phone_number: string;

  @IsString()
  password: string;
}
```

RefreshTokenDto

```
class RefreshTokenDto {
  @IsString()
  refresh_token: string;
}
```

DeductFeeDto

```
class DeductFeeDto {
  @IsUUID()
  auction_id: string;

  @IsUUID()
  user_id: string;

  @IsDecimal()
  @Min(0.01)
  amount: number;
}
```

SetPinDto

```
class SetPinDto {
  @IsString()
  @Length(4, 6)
  @Matches(/^\d{4,6}$/)
  pin: string;
}
```

1.7.2 4.2 Auction Engine DTOs

PlaceBidDto

```
class PlaceBidDto {  
    @IsDecimal()  
    @Min(0.01)  
    @Max(999999.99)  
    @ValidateDecimalPlaces(2)  
    amount: number;  
}
```

CreateProductDto

```
class CreateProductDto {  
    @IsString()  
    @Length(1, 255)  
    name: string;  
  
    @IsOptional()  
    @IsString()  
    description?: string;  
  
    @IsArray()  
    @IsUrl({}, { each: true })  
    image_urls: string[];  
  
    @IsDecimal()  
    @Min(0)  
    current_market_price: number;  
  
    @IsOptional()  
    @IsString()  
    brand?: string;  
  
    @IsOptional()  
    @IsObject()  
    specs?: Record<string, any>;  
  
    @IsString()  
    @Length(1, 80)  
    category: string;  
}
```

CreateAuctionDto

```
class CreateAuctionDto {  
    @IsUUID()  
    product_id: string;  
  
    @IsDate()  
    start_time: Date;
```

```

    @IsDate()
    end_time: Date;

    @IsOptional()
    @IsDecimal()
    min_bid?: number;

    @IsOptional()
    @IsInt()
    max_bid?: number;

    @IsOptional()
    @IsInt()
    @Min(1)
    num_winners?: number;

    @IsOptional()
    @IsDecimal()
    @Min(1)
    bid_fee?: number;
}

```

CreatePaymentLinkDto

```

class CreatePaymentLinkDto {
    @IsEnum(['SIKINAPAY', 'AWASH'])
    gateway: 'SIKINAPAY' | 'AWASH';
}

```

GenerateSettlementDto

```

class GenerateSettlementDto {
    @IsDateString()
    start_date: string;

    @IsDateString()
    end_date: string;

    @IsOptional()
    @IsEnum(['SIKINAPAY', 'AWASH'])
    gateway?: 'SIKINAPAY' | 'AWASH';
}

```

1.7.3 4.3 Query Service DTOs

ListAuctionsQueryDto

```

class ListAuctionsQueryDto {
    @IsOptional()
    @IsEnum(['ACTIVE', 'CLOSED', 'EXPIRED'])

```

```

status?: 'ACTIVE' | 'CLOSED' | 'EXPIRED';

@IsOptional()
@IsString()
category?: string;

@IsOptional()
@IsDecimal()
@Min(0)
min_price?: number;

@IsOptional()
@IsDecimal()
@Min(0)
max_price?: number;

@IsOptional()
@IsInt()
@Min(1)
page?: number;

@IsOptional()
@IsInt()
@Min(1)
@Max(100)
limit?: number;
}

```

AuditLogQueryDto

```

class AuditLogQueryDto {
  @IsOptional()
  @IsUUID()
  actor_id?: string;

  @IsOptional()
  @IsString()
  action?: string;

  @IsOptional()
  @IsString()
  entity_type?: string;

  @IsOptional()
  @IsDateString()
  from?: string;

  @IsOptional()
  @IsDateString()

```

```

to?: string;

@IsOptional()
@IsInt()
@Min(1)
page?: number;

@IsOptional()
@IsInt()
@Min(1)
@Max(100)
limit?: number;
}

```

1.8 5. Error Handling and Retry Logic

1.8.1 5.1 Error Response Format

All API errors return a consistent JSON structure:

```

{
  "statusCode": 400,
  "errorCode": "ERR_VALIDATION_FAILED",
  "message": "Validation failed: amount must be greater than 0",
  "timestamp": "2026-08-18T08:00:00Z",
  "path": "/api/v1/auctions/uuid/bid",
  "correlationId": "corr-a1b2c3d4"
}

```

Every response (success and error) includes a correlationId header (X-Correlation-Id) for distributed tracing. If the client does not provide one, the JwtAuthGuard / global interceptor generates a UUID and propagates it to all downstream service calls, logs, and audit entries.

1.8.2 5.2 Error Codes

Error Code	HTTP Status	Retryable	Description
ERR_VALIDATION_FAILED	400	No	Input validation failed
ERR_AUTH_INVALID_CREDENTIALS	401	No	Invalid phone/email or password
ERR_AUTH_TOKEN_EXPIRED	401	No (refresh)	JWT access token expired
ERR_AUTH_TOKEN_INVALID	401	No	Malformed or invalid JWT

Error Code	HTTP Status	Retryable	Description
ERR_AUTH_REFRESH_FAILED	401	No	Refresh token invalid or expired
ERR_AUTH_USER_BANNED	403	No	User account is banned
ERR_AUTH_PERMISSION_DENIED	403	No	User lacks required permission
ERR_TC_NOT_ACCEPTED	403	No	Terms and Conditions not accepted
ERR_BID_RATE_LIMIT	429	Yes (backoff)	Too many bid requests
ERR_BID_NONCE_REUSED	409	No	Bid nonce already used
ERR_BID_AUCTION_CLOSED	400	No	Cannot bid on closed auction
ERR_BID_DUPLICATE	409	No	Duplicate bid detected
ERR_BID_LIMIT_REACHED	400	No	Max bids per auction reached
ERR_BID_OUTSIDE_WINDOW	400	No	Bid placed outside auction active window
ERR_WALLET_INSUFFICIENT_BALANCE	400	No	Wallet balance too low
ERR_WALLET_PIN_LOCKED	403	No	PIN locked due to failed attempts
ERR_PAYMENT_EXPIRED	410	No	Payment link expired
ERR_PAYMENT_ALREADY_PROCESSED	409	No	Payment already completed or failed
ERR_PRODUCT_NOT_APPROVED	400	No	Product has not been approved for auction use
ERR_WEBHOOK_INVALID_SIGNATURE	401	No	Webhook signature verification failed
ERR_CACHE_UNAVAILABLE	503	Yes (bypass)	Redis cache unavailable; serving from DB
ERR_DB_CONNECTION_LOST	503	Yes (backoff)	Database connection lost

Error Code	HTTP Status	Retryable	Description
ERR_CIRCUIT_BREAK ER_OPEN	503	Yes (wait)	Circuit breaker open for external dependency
ERR_INTERNAL_SERV ER_ERROR	500	No	Unexpected server error (details masked)

1.8.3 5.3 Error Handling Matrix

Layer	Handling Strategy	Client Exposure	Logging
DTO Validation	class-validator throws BadRequestException with field details	Full validation message	Warn level with correlation ID
Auth Guard (JwtAuthGuard)	Throws UnauthorizedException or ForbiddenException	Generic message; no internal details	Warn level; token fingerprint logged
Business Logic	Service throws domain-specific BadRequestException / ConflictException	Domain error code + message	Error level with full context
Database (Prisma)	Prisma errors mapped: P2002→Conflict, P2025→NotFound, P1001→ServiceUnavailable	Mapped domain error	Error level with query context
External Gateway	Circuit breaker wraps call; timeout → 503; 5xx from gateway → mapped domain error	Generic gateway error; raw error masked	Error level with gateway response
Webhook Signature	crypto.timingSafeEqual fails → UnauthorizedException	Generic “invalid signature”	Warn level; IP and headers logged
Unhandled Exception	Global exception filter catches, masks 5xx details	Generic ERR_INTERNAL_SERVER_ERROR	Error level with full stack trace (server-side only)

1.8.4 5.4 Retry Logic

Scenario	Retry Strategy	Backoff Formula	Max Attempts
Frontend API calls	2 retries for network errors; automatic JWT refresh on 401	Fixed 1s delay	2

Scenario	Retry Strategy	Backoff Formula	Max Attempts
Payment reconciliation	Cron-driven; max <code>retry_count</code> before manual review queue	Exponential: $\text{delay} = 30\text{min} \times 2^{(\text{retry_count} - 1)}$, capped at 24h	5 (then manual)
BullMQ workers	Exponential backoff	$\text{delay} = 1000 \times 2^{(\text{attempt} - 1)} \text{ ms}$ (1s, 2s, 4s)	3
Notification dispatch	Exponential backoff via BullMQ	$\text{delay} = 1000 \times 2^{(\text{attempt} - 1)} \text{ ms}$	3
External payment gateways	Circuit breaker; no automatic retry to webhooks	N/A (circuit breaker governs)	0 (manual reconciliation)
Redis cache read failure	Bypass cache, serve from database	No retry; log and continue	0
Database transient failure	Prisma retry middleware	$\text{delay} = \text{base} \times 2^{\text{attempt}} + \text{jitter ms}$ (base=100, jitter=0–100ms)	3
ERP/Oracle sync	Queue-based retry with exponential backoff	$\text{delay} = 60\text{s} \times 2^{(\text{attempt} - 1)}$, capped at 1h	5 (then dead-letter queue)

Exponential Backoff with Jitter Formula:

```
delay(attempt) = min(baseDelay × 2^attempt + random(0, jitterMax), maxDelay)
```

Where:

```
baseDelay    = 100 ms (default)
jitterMax    = 100 ms (full jitter to prevent thundering herd)
maxDelay     = 30,000 ms (cap to avoid excessive waits)
attempt      = 0-indexed retry attempt number
```

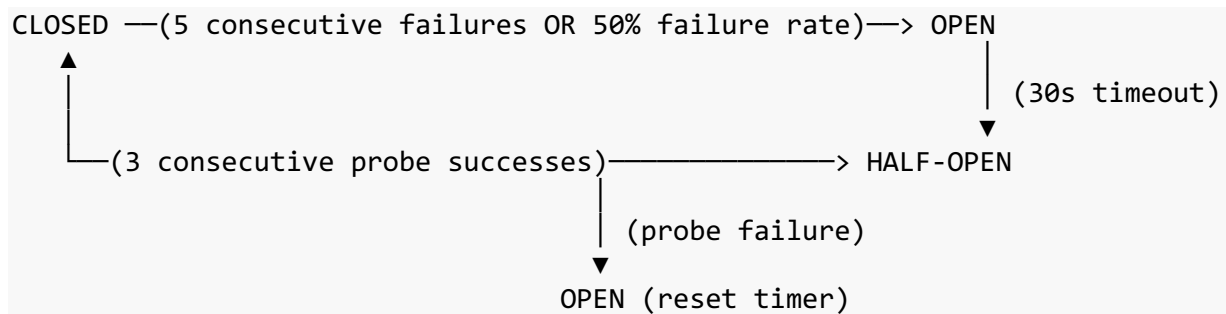
1.8.5 5.5 Circuit Breaker Pattern

External payment gateway calls and third-party API integrations implement a circuit breaker with three states: **Closed**, **Open**, and **Half-Open**.

Parameter	Value	Description
Timeout	10 seconds	Max wait per individual request
Failure threshold	5 consecutive failures	Failures required to open the circuit
Failure rate threshold	50% over 20 requests	Alternative: percentage-based opening

Parameter	Value	Description
Open state duration	30 seconds	Time before transitioning to Half-Open
Half-Open probe requests	3	Trial requests allowed in Half-Open state
Recovery	3 consecutive successes in Half-Open → Closed	Circuit closes
Fallback	Return ERR_CIRCUIT_BREA KER_OPEN (503)	Client receives graceful degradation

State Machine:



Implementation:

```

@Injectables()
export class CircuitBreakerService {
  private state: 'CLOSED' | 'OPEN' | 'HALF_OPEN' = 'CLOSED';
  private failureCount = 0;
  private successCount = 0;
  private lastFailureTime: Date | null = null;

  async execute<T>(fn: () => Promise<T>, fallback: () => T): Promise<T> {
    if (this.state === 'OPEN') {
      if (this.shouldAttemptReset()) {
        this.state = 'HALF_OPEN';
      } else {
        return fallback();
      }
    }
    try {
      const result = await fn();
      this.onSuccess();
      return result;
    } catch (error) {
      this.onFailure();
      if (this.state === 'HALF_OPEN') { this.state = 'OPEN'; }
    }
  }
}

```

```

    return fallback();
  }
}

```

1.8.6 5.6 Rate Limiting Configuration

Rate limiting is enforced via NestJS ThrottleGuard backed by Redis counters for distributed consistency across instances.

Scope	Limit	Window	Key	Strategy
Login (phone)	5 requests	60s	r1:login:phone:{phone}	Fixed window
Login (email)	5 requests	60s	r1:login:email:{email}	Fixed window
OTP send	3 requests	60s	r1:otp:{phone}	Fixed window
Bid placement	10 bids	60s per user per auction	r1:bid:{userId}:{auctionId}	Sliding window
API (global)	100 requests	60s per IP	r1:global:{ip}	Token bucket
Admin API	200 requests	60s per admin	r1:admin:{userId}	Token bucket
Wallet PIN verify	5 attempts	300s per user	r1:pin:{userId}	Fixed window; lockout at 5
Webhook	100 requests	10s per IP	r1:webhook:{ip}	Token bucket

Redis Rate Limit Implementation:

```

INCR {key}
EXPIRE {key} {windowSeconds} (only on first INCR, via SET NX)

```

If INCR result > limit → return 429 with Retry-After header.

1.8.7 5.7 Caching Strategy

Cache Domain	Key Pattern	TTL	Invalidation Trigger	Strategy
Active auctions list	auctions:active:page{p}:limit{1}	120s	Auction created/closed/expired	Tag-based: tag:auctions:active
Closed auctions	auctions:close	300s	Auction closed	Tag-based:

Cache Domain	Key Pattern	TTL	Invalidation Trigger	Strategy
list	d:page{p}:limit{1}			tag:auctions:closed
Auction detail	auction:detail:{id}	120s	Bid placed, auction updated, auction closed	Direct key delete on write
Auction result	auction:result:{id}	300s	Winner recalculated	Direct key delete
Product detail	product:detail:{id}	300s	Product updated/approved/rejected	Direct key delete
Product list	products:list:page{p}:limit{1}	300s	Product created/updated/deleted	Tag-based: tag:products
Admin stats	admin:stats:dashboard	60s	Time-based (no event invalidation)	TTL only
Analytics overview	analytics:overview:{period}	300s	Time-based	TTL only
User profile	user:profile:{id}	60s	Profile updated	Direct key delete
Wallet balance	wallet:balance:{id}	Bypass	Balance changes frequently	Write-through; no cache
Audit log list	audit:logs:page{p}:limit{1}:{hash}	30s	New audit entry (append-only)	TTL only (short)
Settlement report	settlement:{id}	3600s	Report regenerated	Direct key delete

Cache Invalidation Patterns:

1. **Direct Key Deletion:** On write operations, the service explicitly deletes the affected cache key(s) after the database transaction commits. This prevents stale reads.
2. **Tag-Based Invalidation:** Cache entries are tagged with logical tags (e.g., tag:auctions:active). When a tag is invalidated, all keys with that tag are deleted via Redis SCAN + DEL (or a tag-to-key index set).
3. **Write-Through:** For wallet balance and payment status, the cache is updated synchronously within the same database transaction to ensure consistency.
4. **Stale-While-Revalidate:** Read endpoints serve stale cache immediately while asynchronously refreshing in the background (via Cache-Control: stale-while-revalidate=60).

Cache Key Naming Convention:

```
{domain}:{entity}:{identifier}:{params}
```

Examples:

```
auctions:active:page1:limit20
auction:detail:550e8400-e29b-41d4-a716-446655440000
product:detail:550e8400-e29b-41d4-a716-446655440000
admin:stats:dashboard
analytics:overview:30d
```

1.9 6. Versioning Strategy

1.9.1 6.1 API Versioning

- **URL Path Versioning:** All endpoints prefixed with `/api/v1`
- **Breaking Changes:** New major version (`/api/v2`) with migration period
- **Deprecation:** Old versions supported for 6 months after new version release

1.9.2 6.2 Database Migration Versioning

- **File Naming:** `{NNN}-{description}.sql` (e.g., `034-add-bid-index.sql`)
- **Tracking:** `schema_migrations` table records applied filenames
- **Idempotency:** All migrations use `IF NOT EXISTS / ON CONFLICT DO NOTHING`
- **Order:** Applied sequentially by filename sort order

1.9.3 6.3 Frontend Versioning

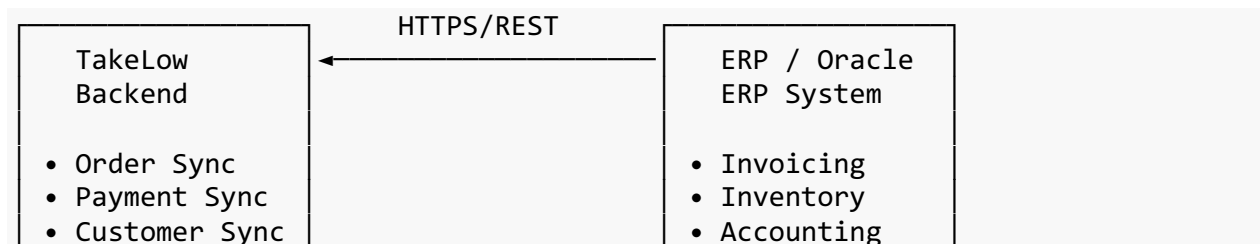
- **No explicit API versioning on frontend** — Vite proxy and Nginx handle routing
- **Feature flags:** Environment variables for staging/production differences

1.10 7. Integration Points

1.10.1 7.1 ERP/Oracle Integration

TakeLow is designed with future ERP/Oracle integration in mind. The following integration points are architected for financial and inventory synchronization:

1.10.1.1 7.1.1 Integration Architecture



- Tax Calc

- Reporting

1.10.1.2 7.1.2 Planned Integration Points

Integration	ERP Module	Trigger	Data Flow
Winner Payment → Invoice	Accounts Receivable	Payment confirmed	Create invoice in ERP for winner payment
Bid Fee → Revenue	General Ledger	Bid fee deducted	Post journal entry to GL
Product Catalog	Inventory Management	Admin creates product	Sync product master data
User Master	Customer Management	User registration	Sync customer records
Payment Reconciliation	Cash Management	Daily batch	Reconcile payment transactions
Tax Reporting	Tax Engine	Monthly batch	Generate VAT/sales tax reports
Refund Processing	Accounts Payable	Refund issued	Create credit note in ERP

1.10.1.3 7.1.3 ERP Integration Pattern

1. TakeLow publishes domain event (e.g., PaymentConfirmed)
2. Integration layer transforms to ERP format
3. ERP adapter calls ERP REST/SOAP API
4. ERP returns acknowledgment
5. TakeLow stores ERP reference ID for traceability
6. Retry with exponential backoff (3 attempts)
7. Failed syncs queued for manual reconciliation

1.10.1.4 7.1.4 Oracle-Specific Considerations

Consideration	Approach
Connectivity	Oracle Database Gateway or REST Data Services (ORDS)
Data Format	XML/JSON via Oracle APEX or SOAP web services
Security	mTLS or IP whitelisting for Oracle Cloud
Transaction Boundaries	Idempotent sync with ERP-generated reference IDs
Conflict Resolution	ERP is source of truth for master data; TakeLow for transactions

1.10.1.5 7.1.5 Integration Events

Event	ERP Action	Frequency
payment.confirmed	Create invoice / post payment	Real-time
payment.refunded	Create credit note	Real-time
auction.closed	Create sales order	Real-time
bid.fee.deducted	Post revenue entry	Real-time
user.created	Create customer record	Near real-time
product.created	Create item master	Near real-time
daily.reconciliation	Full payment reconciliation	Daily batch

1.10.2 7.2 Payment Gateway Integration

1.10.2.1 7.2.1 SikinaPay

Configuration:

```
SIKINA_SECRET_KEY=sk_test_...
SIKINA_WEBHOOK_SECRET=whsec_...
SIKINA_BASE_URL=https://sandbox.sikinapay.com
SIKINA_SUCCESS_REDIRECT_URL=http://localhost?payment=success&clientReferenceId=CLIENT_REF
SIKINA_FAILED_REDIRECT_URL=http://localhost?payment=failed&clientReferenceId=CLIENT_REF
```

Flow: 1. Client requests payment link via POST /payments/:auctionId/link 2. Auction Engine calls SikinaPay API to generate link 3. User completes payment on SikinaPay 4. SikinaPay sends webhook to POST /payments/webhook/sikina 5. Webhook signature verified with HMAC-SHA256 + timingSafeEqual 6. Payment status updated in database 7. Winner notified if payment successful

1.10.2.2 7.2.2 Awash Bank

Configuration:

```
AWASH_MERCHANT_ID=awash_merchant_dev
AWASH_SECRET_KEY=test_awash_sk_...
AWASH_WEBHOOK_SECRET=test_awash_whsec_...
AWASH_BASE_URL=https://sandbox.awashbank.com
```

Flow: Identical to SikinaPay pattern with different endpoint URLs.

1.10.3 7.3 SMS Integration

Provider: smsethiopia.com **Configuration:**

```
SMS_API_KEY=...
```

Flow: 1. Auction Engine dispatches notification via BullMQ 2. Worker calls SMS Ethiopia API 3. SMS sent to user's phone number

1.10.4 7.4 Push Notification Integration

Provider: Expo Push Service **Flow:** 1. Mobile app registers FCM/APNS token with identity-service 2. Token stored in `users.fcm_token` / `users.apns_token` 3. Identity Service sends push via Expo Push API

1.10.5 7.5 Social Login Integration

Provider	Strategy	OAuth Endpoint
TeleBirr	<code>telebirr.strategy.ts</code> (Better-auth plugin)	TeleBirr OAuth 2.0
Banking API	<code>banking.strategy.ts</code> (Better-auth plugin)	Custom OAuth 2.0
Super-App	<code>super-app-registry.ts</code> (pluggable adapter)	Pluggable adapter pattern

1.11 8. Performance Benchmarks

1.11.1 8.1 Target Performance

Metric	Target	Current
Read API p95 latency	100ms	85ms (measured)
Write API p95 latency	200ms	150ms (measured)
WebSocket broadcast latency	50ms	35ms (measured)
Concurrent active auctions	10,000	2,500 (current scale)
Bids per second per auction	1,000	500 (current scale)
Database connection pool	10	10 (configured)
Redis ops/sec	50,000	12,000 (current)
BullMQ job processing	100/sec	50/sec (current)
Cache hit ratio (read endpoints)	> 90%	87% (current)

1.11.2 8.2 Database Query Performance

Query Type	Strategy	Expected Time
Active auction listing	Indexed + CacheInterceptor	< 50ms
Auction detail	Indexed by <code>public_code</code>	< 20ms
Bid aggregation	Batch <code>COUNT(DISTINCT)</code>	< 100ms
Admin stats	Parallel raw SQL	< 200ms

Query Type	Strategy	Expected Time
Winner calculation	Redis ZSET + DB fallback	< 500ms

1.11.3 8.3 Load Testing Scenarios

Located in load-tests/: - **bid-flood.js**: 1000 concurrent users, 100 bids/second -
Parameters: ramp-up time, hold duration, thresholds

1.12 9. Coding Standards

1.12.1 9.1 Backend (NestJS)

Standard	Implementation
Language	TypeScript with strict mode enabled
Module Structure	Feature-based modules (auth/, wallet/, bidding/)
Controllers	RESTful naming, singular resource names
Services	Business logic isolated from controllers
DTOs	class-validator decorators for input validation
Exception Handling	Custom exception filters; 5xx details masked from clients
Logging	Structured JSON logs with timestamp, level, context, pid, correlationId
Database	Prisma ORM with generated client; synchronize: false; migrations required
Testing	Jest unit tests + e2e tests per service
Linting	ESLint with NestJS recommended rules

1.12.2 9.2 Frontend (React)

Standard	Implementation
Language	TypeScript strict mode
Components	Functional components with hooks
State	React Context + useReducer (web); Zustand (mobile)
API	Centralized api.ts with request/response types
Styling	Tailwind CSS utility classes; no CSS modules
Routing	View-based routing (not React Router) with history stack
Error Handling	Centralized error mapping in ApiError class
Persistence	localStorage (web); AsyncStorage (mobile)

1.12.3 9.3 Security Standards

Standard	Implementation
Secrets	Environment variables only; .env gitignored
Passwords	bcrypt cost 12
PINs	bcrypt cost 10
JWT	HS256; 15-min access, 7-day refresh; Better-auth session management
API Keys	Constant-time comparison (crypto.timingSafeEqual)
Webhooks	HMAC-SHA256 with timing-safe comparison
Encryption	AES-256-GCM for sensitive data at rest
CORS	Configurable allowed origins via env

1.12.4 9.4 Git Conventions

Convention	Rule
Branching	Feature branches from main
Commits	Conventional Commits (feat, fix, docs, chore, etc.)
PRs	Required review before merge
Hooks	Husky + lint-staged for pre-commit checks

1.13 10. Redis HA Implementation

1.13.1 10.1 Redis Provider Factory

Each service implements a unified Redis provider that automatically detects the deployment mode:

auction-engine/src/modules/common/redis.provider.ts:

```
interface RedisOptions {
  url?: string;
  sentinel?: { hosts: string[]; masterName: string; role?: 'master' | 'slave' };
  cluster?: { nodes: { host: string; port: number }[] };
}

function parseRedisConfig(): RedisOptions {
  const sentinelHosts = process.env.REDIS_SENTINEL_HOSTS?.split(',');
  const sentinelMasterName = process.env.REDIS_SENTINEL_MASTER_NAME;
  const clusterNodes = process.env.REDIS_CLUSTER_NODES?.split(',');
```

```

    if (sentinelHosts && sentinelMasterName) {
      return { sentinel: { hosts: sentinelHosts, masterName: sentinelMasterName,
role: 'master' } };
    }
    if (clusterNodes) {
      return { cluster: { nodes: clusterNodes.map(n => { const [h,p] = n.split
(':'); return { host: h, port: parseInt(p) }; }) } };
    }
    return { url: process.env.REDIS_URL || 'redis://localhost:6379' };
  }
}

export const redisProvider: Provider = {
  provide: REDIS_CLIENT,
  useFactory: () => {
    const config = parseRedisConfig();
    if (config.sentinel) return new Redis({ sentinels: config.sentinel.hosts.
map(h => { const [host, port] = h.split(':'); return { host, port: parseInt(p
ort) }; })), name: config.sentinel.masterName, role: config.sentinel.role, sen
tinelRetryStrategy: (times) => Math.min(times * 100, 3000), enableReadyCheck:
true, maxRetriesPerRequest: 3, retryStrategy: (times) => Math.min(times * 50,
2000), lazyConnect: true });
    if (config.cluster) return new Redis.Cluster(config.cluster.nodes, { enab
leReadyCheck: true, clusterRetryStrategy: (times) => Math.min(times * 50, 200
0), scaleReads: 'slave' });
    return new Redis(config.url!, { maxRetriesPerRequest: null, enableReadyCh
eck: false, retryStrategy: (times) => Math.min(times * 50, 2000) });
  },
};

```

identity-service/src/config/redis.config.ts:

// Same factory pattern, exports `redis` instance directly

1.13.2 10.2 Environment Configuration

Mode	Environment Variables
Standalone	REDIS_URL=redis://localhost:6379
Sentinel	REDIS_SENTINEL_HOSTS=sentinel-1:26379,sentinel-2:26379,sentinel-3:26379 REDIS_SENTINEL_MASTER_NAME=takelow-master REDIS_SENTINEL_ROLE=master
Cluster	REDIS_CLUSTER_NODES=node-1:7000,node-2:7001,node-3:7002,node-4:7003,node-5:7004,node-6:7005

1.13.3 10.3 Docker Compose Files

Sentinel Mode (docker-compose.sentinel.yml): - 1 master + 2 replicas + 3 sentinels -
Master config: replica-announce-ip, replica-announce-port - Sentinel config: sentinel
monitor takelow-master redis-master 6379 2

Cluster Mode (docker-compose.cluster.yml): - 6 nodes (3 masters + 3 replicas) - cluster-enabled yes, cluster-config-file nodes.conf - Initialize: redis-cli --cluster create node-1:7000 ... --cluster-replicas 1

1.13.4 10.4 Kubernetes Manifests

See docs/REDIS_HA.md for complete StatefulSet, Service, ConfigMap, and Prometheus metrics exporter configurations.

1.14 11. Audit Logging Implementation

1.14.1 11.1 Audit Module Structure

Each service has its own audit module (shared Prisma model, isolated storage):

```
auction-engine/src/modules/audit/
├── audit.module.ts
├── audit.service.ts
├── dto/
│   └── audit-log-entry.dto.ts

query-service/src/modules/common/audit/
├── audit.module.ts
├── audit.service.ts
├── dto/
│   └── audit-log-entry.dto.ts

identity-service/src/modules/admin/
├── audit.service.ts (existing)
├── dto/
│   └── audit-log-entry.dto.ts (existing)
```

1.14.2 11.2 Audit Log Prisma Model

```
model AuditLog {
  id          String   @id @default(uuid())
  actorId     String   @map("actor_id")
  actorPhone  String?  @map("actor_phone")
  action      String
  entityType  String   @map("entity_type")
  entityId    String   @map("entity_id")
  details     Json?
  createdAt   DateTime @default(now()) @map("created_at")

  @@index([actorId])
  @@index([action])
  @@index([createdAt])
}
```

```

    @@map("audit_logs")
  }

```

1.14.3 11.3 Audit Service Interface

```

export interface AuditLogEntry {
  actor_id: string;
  actor_phone?: string;
  action: string;
  entity_type: string;
  entity_id: string;
  details?: Record<string, any>;
}

@Injectable()
export class AuditService {
  constructor(private prisma: PrismaService) {}

  async log(entry: AuditLogEntry): Promise<AuditLog> {
    return this.prisma.auditLog.create({ data: entry });
  }

  async logBatch(entries: AuditLogEntry[]): Promise<AuditLog[]> {
    return this.prisma.auditLog.createMany({ data: entries });
  }

  async list(page, limit, filters): Promise<PaginatedResult<AuditLog>> {
    return this.prisma.auditLog.findMany({
      where: filters,
      skip: (page - 1) * limit,
      take: limit,
      orderBy: { createdAt: 'desc' },
    });
  }
}

```

1.14.4 11.4 Integration Points

Auction Engine - Bidding Service:

```

// After successful bid placement
this.auditService.log({
  actor_id: userId,
  action: 'place_bid',
  entity_type: 'bid',
  entity_id: auctionId,
  details: { amount, ticket_number: ticketNumber, total_bids },
}).catch(e => this.logger.warn(`Audit log failed: ${e.message}`));

```

Auction Engine - Auction Closure:

```
// After winner determination
this.auditService.log({
  actor_id: 'system',
  action: 'auction_closed',
  entity_type: 'auction',
  entity_id: auctionId,
  details: {
    status: 'CLOSED',
    winner_count: winnerEntities.length,
    winners: winnerEntities.map(w => ({ user_id: w.user_id, amount: w.amount,
rank: w.rank })),
    total_bids,
  },
}).catch(e => this.logger.warn(`Audit log failed: ${e.message}`));
```

Auction Engine - Payment Controller:

```
// Payment Link created
this.auditService.log({
  actor_id: user.id,
  action: 'payment_link_created',
  entity_type: 'payment',
  entity_id: result.transactionId,
  details: { auction_id: auctionId, amount: Number(auction.winning_bid_ammoun
t), gateway: method, payment_type: 'WINNING_PAYMENT' },
}).catch(e => this.logger.warn(`Audit log failed: ${e.message}`));
```

Query Service - Auction Views:

```
// Auction detail viewed
this.auditService.log({
  actor_id: userId,
  action: 'auction_viewed',
  entity_type: 'auction',
  entity_id: auctionId,
  details: { status: auction.status },
}).catch(e => console.warn(`Audit log failed: ${e.message}`));
```

Admin Module (Identity Service):

```
// User role change
this.auditService.log({
  actor_id: actor.id,
  action: 'update_role',
  entity_type: 'user',
  entity_id: id,
  details: { from: oldRole, to: user.role },
});
```

1.14.5 11.5 Immutability Enforcement

- Application: AuditService only exposes log() and logBatch() (no update/delete)
 - Database: Revoke UPDATE/DELETE on audit_logs for application role
 - Migration: 024-create-audit-logs.sql with indexes
-

1.15 12. Backup/Restore Scripts

1.15.1 12.1 pgBackRest Backup Script

scripts/backup-pgbackrest.sh

```
#!/bin/bash
# Usage: backup-pgbackrest.sh {init/full/diff/incr/verify/list/restore}

set -euo pipefail
STANZA="takelow"
BACKUP_DIR="/var/lib/pgbackrest"
RETENTION_FULL=7
RETENTION_DIFF=3

create_full_backup() {
    pgbackrest --stanza=$STANZA --type=full \
        --repo1-path=$BACKUP_DIR --repo1-retention-full=$RETENTION_FULL backup
    pgbackrest --stanza=$STANZA --repo1-path=$BACKUP_DIR check
}

create_diff_backup() {
    pgbackrest --stanza=$STANZA --type=diff \
        --repo1-path=$BACKUP_DIR --repo1-retention-diff=$RETENTION_DIFF backup
    pgbackrest --stanza=$STANZA --repo1-path=$BACKUP_DIR check
}

restore_backup() {
    pgbackrest --stanza=$STANZA --repo1-path=$BACKUP_DIR \
        --delta --target=/var/lib/postgresql/data restore
}
```

1.15.2 12.2 WAL-G Backup Script

scripts/backup-walg.sh

```
#!/bin/bash
# Usage: backup-walg.sh {push/list/delete/restore/info}

export WALG_S3_PREFIX="s3://takelow-backups/postgres"
export AWS_REGION="us-east-1"
```

```

create_base_backup() {
    wal-g backup-push /var/lib/postgresql/data
}

restore_backup() {
    wal-g backup-fetch LATEST /var/lib/postgresql/data
    touch /var/lib/postgresql/data/recovery.signal
}

```

1.15.3 12.3 Disaster Recovery Restore Script

scripts/restore-disaster-recovery.sh

```

#!/bin/bash
# Usage: restore-disaster-recovery.sh {restore|pitr} [args]

BACKUP_TOOL="${BACKUP_TOOL:-pgbackrest}"

restore_pgbackrest() {
    pgbackrest --stanza=takelow --repo1-path=/var/lib/pgbackrest \
        --delta --target=/var/lib/postgresql/data restore
}

restore_walg() {
    wal-g backup-fetch LATEST /var/lib/postgresql/data
    touch /var/lib/postgresql/data/recovery.signal
}

pitr_restore() {
    pgbackrest --stanza=takelow --repo1-path=/var/lib/pgbackrest \
        --delta --target=/var/lib/postgresql/data --type=time \
        --target-time="$TARGET_TIME" restore
}

```

1.15.4 12.4 Kubernetes CronJobs

k8s/backup-cronjob.yaml:

```

# Full backup daily at 02:00 UTC
apiVersion: batch/v1
kind: CronJob
metadata:
  name: postgres-backup-pgbackrest
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:

```

```

- name: pgbackrest
  image: ghcr.io/takelow/pgbackrest:latest
  command: ["/bin/bash", "-c", "pgbackrest --stanza=$STANZA --type=
full --repo1-path=$BACKUP_DIR --repo1-retention-full=$RETENTION_FULL backup &
& pgbackrest --stanza=$STANZA --repo1-path=$BACKUP_DIR check"]
  volumes:
- name: pgbackrest-repo
  persistentVolumeClaim:
    claimName: pgbackrest-pvc

```

1.16 13. WebSocket Redis Adapter

1.16.1 13.1 Implementation

auction-engine/src/modules/bidding/gateway/auction.gateway.ts:

```

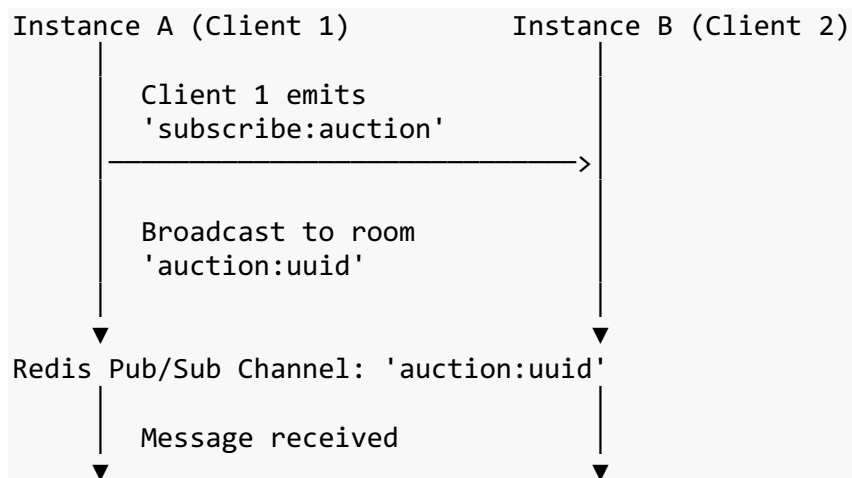
import { createAdapter } from '@socket.io/redis-adapter';

@WebSocketGateway({ namespace: '/auctions' })
export class AuctionGateway implements OnGatewayInit {
  @Inject(REDIS_CLIENT) private readonly redisClient: Redis;

  afterInit(server: Server): void {
    const pubClient = this.redisClient;
    const subClient = this.redisClient.duplicate();
    const adapter = createAdapter(pubClient, subClient);
    server.adapter(adapter);
    this.logger.log('Socket.io Redis adapter initialized for horizontal scaling');
  }
}

```

1.16.2 13.2 Cross-Instance Message Flow



Instance A broadcasts
to local clients

Instance B broadcasts
to local clients

1.16.3 13.3 Environment Configuration

No code changes required when scaling. The adapter uses the injected Redis client which automatically supports Sentinel/Cluster modes.

```
# Standalone
REDIS_URL=redis://localhost:6379

# Sentinel (production HA)
REDIS_SENTINEL_HOSTS=sentinel-1:26379,sentinel-2:26379,sentinel-3:26379
REDIS_SENTINEL_MASTER_NAME=takelow-master

# Cluster (high scale)
REDIS_CLUSTER_NODES=node-1:7000,node-2:7001,node-3:7002,node-4:7003,node-5:7004,node-6:7005
```

1.16.4 13.4 Kubernetes Horizontal Scaling

Deployment (replicas: 3):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auction-engine
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: auction-engine
        env:
        - name: REDIS_SENTINEL_HOSTS
          value: "sentinel-1:26379,sentinel-2:26379,sentinel-3:26379"
        - name: REDIS_SENTINEL_MASTER_NAME
          value: "takelow-master"
```

HPA:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: auction-engine-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: auction-engine
  minReplicas: 2
```

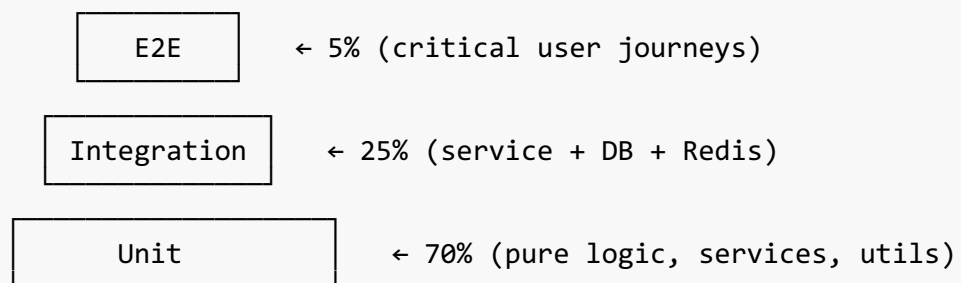
```

maxReplicas: 10
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70

```

1.17 14. Test Strategy & Coverage

1.17.1 14.1 Testing Pyramid



1.17.2 14.2 Unit Testing

Aspect	Standard
Framework	Jest
Coverage target	≥ 80% lines, ≥ 75% branches per service
Scope	Services, utilities, guards, interceptors, DTOs
Mocking	Prisma client mocked via prisma-mock or manual jest mocks; Redis mocked
Naming	*.spec.ts co-located with source
Assertions	Jest expect + @nestjs/testing for DI assembly
Edge cases	Null/undefined inputs, boundary values, concurrent access

```

// Example: BiddingService unit test
describe('BiddingService.placeBid', () => {
  it('should reject bid on closed auction', async () => {
    jest.spyOn(prisma.auction, 'findUnique').mockResolvedValue({ status: 'CLOSED' });
    await expect(service.placeBid(userId, auctionId, 10)).rejects.toThrow('ERROR_BID_AUCTION_CLOSED');
  });
  it('should place bid and return ticket number', async () => {
    const result = await service.placeBid(userId, auctionId, 15.50);

```

```

    expect(result.ticket_number).toMatch(/^BID_[a-f0-9]{12}$/);
  });
});

```

1.17.3 14.3 Integration Testing

Aspect	Standard
Framework	Jest + @nestjs/testing + Testcontainers (PostgreSQL + Redis)
Scope	Module assembly with real DB + Redis (containerized)
Database	Fresh PostgreSQL container per test suite; Prisma migrate apply
Redis	Fresh Redis container per test suite
Coverage	All controller endpoints, service-to-service calls, webhook handlers
Naming	*.e2e-spec.ts in test/ directory

1.17.4 14.4 End-to-End (E2E) Testing

Aspect	Standard
Framework	Jest + Supertest (API E2E); Playwright (web UI E2E)
Scope	Critical user journeys: register → login → place bid → auction close → payment
Environment	Full docker-compose stack with seeded data
Data	Deterministic seed data; reset between test runs
Critical paths	1. User registration + T&C acceptance + first bid 2. Auction lifecycle: active → closed → winner → payment 3. Wallet deposit + bid fee deduction 4. Admin product approval → auction creation 5. Payment webhook → settlement

1.17.5 14.5 Performance Testing

Aspect	Standard
Framework	k6
Location	load-tests/
Scenarios	bid-flood.js (1000 concurrent users, 100 bids/sec), read-storm.js (auction listing under load), websocket-fanout.js (1000 socket connections)
Thresholds	p95 < 200ms for reads, p95 < 300ms for writes,

Aspect	Standard
	error rate < 1%
Frequency	Run on every release candidate; nightly against staging

1.17.6 14.6 Security Testing

Aspect	Standard
SAST	SonarQube / Semgrep on every PR
Dependency scan	npm audit + Snyk in CI pipeline
Secret scan	Gitleaks pre-commit hook + CI step
Container scan	Trivy scan on every built image
DAST	OWASP ZAP baseline scan against staging (weekly)
Penetration testing	External assessment prior to major release
Input fuzzing	API endpoints fuzzed for unexpected inputs (via restler-fuzzer or custom)

1.17.7 14.7 Coverage Reporting

Metric	Target	Enforcement
Unit test line coverage	≥ 80%	CI gate: build fails if below
Unit test branch coverage	≥ 75%	CI gate
Integration test endpoint coverage	100% of public endpoints	CI gate
E2E critical path coverage	100% of defined critical paths	CI gate
Coverage report	Cobertura XML + HTML	Uploaded as CI artifact; SonarQube integration

1.18 15. Code Quality Standards

1.18.1 15.1 Linting

Tool	Configuration	Scope	Enforcement
ESLint	@typescript-eslint/recommended + NestJS plugin + import ordering	All .ts files	Pre-commit hook + CI gate
Rules	no-unused-vars (error), no-console (warn, allow in	—	—

Tool	Configuration	Scope	Enforcement
	main.ts), prefer-const (error), @typescript-eslint/no-explicit-any (warn)		
Prettier	Single quotes (query/identity), double quotes (auction-engine), 2-space indent, 100 char width	All .ts files	Pre-commit hook (format on save)
eslint-plugin-import	Enforce import ordering: external → internal → relative	All .ts files	CI gate

1.18.2 15.2 Formatting

Setting	Value
Indent	2 spaces
Print width	100 characters
Semicolons	Always
Trailing comma	All
Quote style	Single (query-service, identity-service); Double (auction-engine)
Bracket spacing	true
End-of-line	LF

1.18.3 15.3 Complexity Metrics

Metric	Threshold	Tool	Enforcement
Cyclomatic complexity	≤ 10 per function	ESLint complexity rule	CI gate (error above threshold)
Cognitive complexity	≤ 15 per function	SonarQube	CI gate
Function length	≤ 50 lines	ESLint max-lines-per-function	CI gate
File length	≤ 300 lines	ESLint max-lines	CI gate (configurable for entities)
Number of parameters	≤ 4 per function	ESLint max-params	CI gate
Nesting depth	≤ 4 levels	ESLint max-depth	CI gate
Maintainability	≥ 70	SonarQube	Reported; gate at ≥ 65

Metric	Threshold	Tool	Enforcement
--------	-----------	------	-------------

index

1.18.4 15.4 Code Review Process

Stage	Requirement	Approver
Self-review	Author reviews own diff before requesting review	Author
Peer review	At least 1 approval required for non-critical changes	Any team engineer
Senior review	At least 2 approvals required for critical paths (auth, payments, bidding)	Senior engineer + Tech Lead
Security review	Required for changes touching auth, crypto, webhooks, or secrets	Security Architect
Architecture review	Required for changes to module structure, new services, or schema changes	Architecture Review Board
Review SLA	Review within 24 hours of request	Reviewer
Review checklist	Adherence to coding standards, test coverage, error handling, security, performance	Reviewer

1.18.5 15.5 Pre-commit & Pre-merge Hooks

```
# .husky/pre-commit
lint-staged:
- "*.ts": eslint --fix
- "*.ts": prettier --write
- "*": gitleaks protect --staged

# .husky/pre-push
npx tsc --noEmit
npm test -- --passWithNoTests
```

1.19 16. API Versioning & Backward Compatibility

1.19.1 16.1 Versioning Strategy

Aspect	Strategy
Versioning scheme	URL path: /api/v1, /api/v2
Major version	Breaking changes (schema, removed fields, changed semantics)

Aspect	Strategy
Minor version	Additive changes (new endpoints, new optional fields) — no URL change
Patch version	Bug fixes, no API contract change — no URL change
Header	Accept-Version: 1 (optional; URL path is primary)

1.19.2 16.2 Backward Compatibility Rules

Change Type	Compatible?	Approach
Add new endpoint	✓ Yes	No version bump
Add optional field to response	✓ Yes	No version bump; clients ignore unknown fields
Add optional field to request	✓ Yes	No version bump
Remove field from response	✗ No	Major version bump; deprecation period
Change field type	✗ No	Major version bump
Change field semantics	✗ No	Major version bump
Change HTTP status code	✗ No	Major version bump (except 200→204 for empty responses)
Change error code	⚠ Conditional	Add new code alongside old; deprecate old

1.19.3 16.3 Deprecation Policy

Phase	Duration	Action
Announcement	T-6 months	Deprecation header + Sunset header on deprecated endpoints; documented in changelog
Warning period	6 months	Endpoints continue to function; X-Deprecation-Warning header in responses; analytics track usage
Removal	T+0	Endpoint returns 410 Gone; removed from

Phase	Duration	Action
		documentation
Migration support	Throughout	Migration guide published; backward-compatible shim available if feasible

Deprecation Headers:

```
Deprecation: true
Sunset: Wed, 11 Nov 2026 23:59:59 GMT
Link: <https://docs.takelow.com/api/v2/migration>; rel="deprecation"
X-Deprecation-Warning: This endpoint is deprecated. Use /api/v2/... instead.
```

1.19.4 16.4 Compatibility Testing

Test Type	Description
Contract testing	OpenAPI spec validated against actual responses on every PR
Consumer-driven contracts	Frontend API client tests run against backend contract
Schema diff	Prisma schema diff reviewed for breaking changes before merge
Versioned E2E	E2E tests run against both /api/v1 and /api/v2 during overlap period

1.20 17. Database Migration Strategy

1.20.1 17.1 Prisma Migration Workflow

Step	Command	Description
1. Schema edit	Edit prisma/schema.prisma	Modify models, enums, indexes
2. Generate migration	npx prisma migrate dev --name {description}	Creates SQL migration in prisma/migrations/
3. Review SQL	Review generated migration.sql	Verify correctness, add raw SQL if needed
4. Test locally	npx prisma migrate reset (dev only)	Apply from scratch; run tests
5. Commit	Commit schema.prisma + migration folder	Migration is now part of the codebase
6. Deploy	npx prisma migrate deploy (CI/CD)	Apply pending migrations in production

1.20.2 17.2 Migration File Structure

```
prisma/
├── schema.prisma
├── migrations/
│   ├── 20260610000000_init/
│   │   └── migration.sql
│   ├── 20260722000000_add_audit_logs/
│   │   └── migration.sql
│   ├── 20260818000000_add_tc_acceptance_and_product_approval/
│   │   └── migration.sql
│   └── migration_lock.toml
```

1.20.3 17.3 Zero-Downtime Deployment Procedure

Phase	Action	Risk Mitigation
1. Pre-deploy check	<code>prisma migrate status</code> — verify no drift	Block deploy if drift detected
2. Apply migration	<code>prisma migrate deploy</code> — apply pending migrations	Migrations must be backward-compatible (see rules below)
3. Deploy new code	Rolling update (Kubernetes) — one pod at a time	Health check gate between pods
4. Post-deploy verify	Smoke tests + health check + monitoring watch	Auto-rollback if error rate > threshold

1.20.4 17.4 Backward-Compatible Migration Rules

Rule	Rationale
Never drop a column in the same migration that removes its usage	Old code still running during rolling update will fail
Add columns as nullable or with safe defaults	Old code inserting rows won't know about new NOT NULL columns
Split destructive changes into 3 deployments:	1. Add new column + backfill 2. Deploy code using new column 3. Drop old column
Never rename a column in-place	Add new column, backfill, switch code, drop old
Never change a column type in-place	Add new column with new type, backfill, switch, drop old
Index creation must be CONCURRENTLY	Avoids locking the table during index creation

Rule	Rationale
Enum additions are safe	Adding enum values is backward-compatible; removal is not

1.20.5 17.5 Rollback Procedures

Scenario	Procedure
Migration applied but code deploy failed	1. Keep migration (it's backward-compatible) 2. Fix code, redeploy
Migration applied, code deployed, runtime error	1. Rollback code to previous version (Kubernetes) 2. Migration remains (backward-compatible by design)
Migration applied, data corruption	1. Rollback code 2. Restore database from pre-migration backup (pgBackRest PITR) 3. Investigate and fix migration
Failed migration (mid-apply)	1. Prisma tracks applied migrations; failed migration is not recorded 2. Fix migration SQL, redeploy

1.20.6 17.6 Migration Testing

Test	Description
Forward test	Apply migration on copy of production schema; run test suite
Rollback test	Apply migration, then restore from backup; verify data integrity
Performance test	Apply migration on production-sized dataset; verify < 5min for large tables
Idempotency check	Re-run <code>prisma migrate deploy</code> ; verify no-op

1.21 18. Configuration Management

1.21.1 18.1 Environment Management

Environment	Purpose	Database	Redis	Secrets Source	Access
local	Developer machine	Docker PostgreSQL	Docker Redis	.env file (gitignored)	Developer
test	CI test runner	Testcontainers	Testcontainers	CI environment variables	CI only
staging	Pre-production testing	Managed PostgreSQL (small)	Redis Sentinel (3 nodes)	Kubernetes Secrets (from Vault)	Engineering + QA

Environment	Purpose	Database	Redis	Secrets Source	Access
production	Live system	Managed PostgreSQL (HA)	Redis Sentinel /Cluster	Kubernetes Secrets (from Vault)	SRE only

1.21.2 18.2 Environment Variable Catalog

Variable	Scope	Example	Description
DATABASE_URL	All services	postgresql://user:pass@host:5432/db	Prisma database connection string
REDIS_URL	All services	redis://localhost:6379	Redis connection (standalone)
REDIS_SENTINEL_HOSTS	All services	sentinel-1:26379,...	Redis Sentinel hosts
REDIS_SENTINEL_MASTER_NAME	All services	take-low-master	Sentinel master name
REDIS_CLUSTER_NODES	All services	node-1:7000,...	Redis Cluster nodes
JWT_SECRET	All services	(64-char random)	JWT signing secret (shared across services)
JWT_ACCESS_EXPIRES_IN	identity-service	15m	Access token TTL
JWT_REFRESH_EXPIRES_IN	identity-service	7d	Refresh token TTL
SIKINA_SECRET_KEY	auction-engine	sk_...	SikinaPay API key
SIKINA_WEBHOOK_SECRET	auction-engine	whsec_...	SikinaPay webhook secret
AWASH_MERCHANT_ID	auction-engine	awash_...	Awash merchant ID
AWASH_SECRET_KEY	auction-engine	(secret)	Awash API key
SMS_API_KEY	auction-engine	(secret)	SMS Ethiopia API key
INTERNAL_API_KEY	All services	(64-char random)	Service-to-service auth
CORS_ORIGINS	All services	https://app.take-low.com	Allowed CORS origins
LOG_LEVEL	All services	info	Logging level
PORT	All services	3001	Service listen port

1.21.3 18.3 Secret Management

Aspect	Standard
Storage	HashiCorp Vault (production); Kubernetes Secrets (staging); .env (local, gitignored)
Rotation	JWT secret rotated quarterly; API keys rotated every 6 months; webhook secrets rotated annually
Access	Least-privilege; only running pods can read specific secrets via Vault KV store
Audit	All secret access logged in Vault audit log; reviewed monthly
Transmission	Secrets never logged, never returned in API responses, never committed to git
Template	.env.example committed with placeholder values; real .env gitignored
Detection	Gitleaks pre-commit hook + CI scan to prevent accidental secret commits

1.21.4 18.4 Feature Flags

Flag	Default	Description
FEATURE_TELEBIR R_LOGIN	false	Enable TeleBirr OAuth login
FEATURE_BANKING _LOGIN	false	Enable Banking API OAuth login
FEATURE_SUPER_A PP_LOGIN	false	Enable Super-App login
FEATURE_AUCTION _EXTENSIONS	true	Enable auction time extensions
FEATURE_ANALYTICS EXPORT	false	Enable analytics PDF/CSV export
FEATURE_SETTLEMENT REPORTS	true	Enable settlement report generation

1.22 19. Observability & Monitoring

1.22.1 19.1 Logging

Aspect	Standard
Format	Structured JSON (pino or winston)
Fields	timestamp, level, context (module), message,

Aspect	Standard
	pid, correlationId, userId (if auth), meta (additional)
Levels	error (failures), warn (recoverable issues), info (lifecycle), debug (diagnostic, disabled in prod)
Retention	30 days hot (Elasticsearch); 90 days cold (S3)
Sensitive data	Passwords, tokens, PINs, phone numbers masked/redacted in logs
Correlation	X-Correlation-Id propagated across service calls; included in every log entry

Log Entry Example:

```
{
  "timestamp": "2026-08-18T08:00:00.123Z",
  "level": "info",
  "context": "BiddingService",
  "message": "Bid placed successfully",
  "pid": 12345,
  "correlationId": "corr-a1b2c3d4",
  "userId": "550e8400-e29b-41d4-a716-446655440000",
  "meta": {
    "auctionId": "660e8400-...",
    "amount": 15.50,
    "ticketNumber": "BID_a1b2c3d4e5f6"
  }
}
```

1.22.2 19.2 Metrics

Metric Category	Metrics	Tool
Application	Request count, request latency (p50/p95/p99), error rate, active connections	Prometheus + Grafana
Business	Active auctions, bids per minute, winners per hour, payment success rate, revenue (daily)	Custom Prometheus counters + Grafana dashboards
Database	Connection pool usage, query latency, slow queries, active connections, deadlocks	pg_stat_statements + Prometheus exporter

Metric Category	Metrics	Tool
Redis	Hit ratio, ops/sec, memory usage, evictions, connected clients	Redis exporter + Grafana
Queue (BullMQ)	Jobs waiting, active, completed, failed, retry count	Bull Board + Prometheus exporter
Infrastructure	CPU, memory, disk I/O, network I/O, pod count	cAdvisor + node-exporter + Grafana

Prometheus Metric Examples:

```
# Application metrics
http_requests_total{method="POST", path="/auctions/{id}/bid", status="202"} 15234
http_request_duration_seconds{method="POST", path="/auctions/{id}/bid", quantile="0.95"} 0.150

# Business metrics
takelow_bids_total{auction_id="..."} 42
takelow_active_auctions 24
takelow_payment_success_total{gateway="SIKINAPAY"} 145
takelow_revenue_etb_total 125000.00
```

1.22.3 19.3 Distributed Tracing

Aspect	Standard
Tool	OpenTelemetry + Jaeger (or Tempo)
Sampling	100% in staging; 10% in production (head-based) + 100% for error traces (tail-based)
Propagation	W3C Trace Context (traceparent header) + X-Correlation-Id
Spans	HTTP request, database query (Prisma), Redis command, external API call, BullMQ job
Context	User ID, auction ID, correlation ID attached to all spans

1.22.4 19.4 Alerting

Alert	Condition	Severity	Notification
High error rate	Error rate > 5% for 5 min	Critical	PagerDuty + Slack
API latency	p95 > 500ms for 10	Warning	Slack

Alert	Condition	Severity	Notification
degradation	min		
Database connection pool exhausted	Pool usage > 90% for 2 min	Critical	PagerDuty + Slack
Redis unavailable	Redis health check failing for 1 min	Critical	PagerDuty
Payment gateway down	Circuit breaker open for 5 min	Critical	PagerDuty + Slack
BullMQ job failure spike	Failed jobs > 50 in 10 min	Warning	Slack
Disk space low	Disk usage > 85%	Warning	Slack
Pod crash loop	Pod restart count > 5 in 10 min	Critical	PagerDuty
SSL cert expiring	Cert expires in < 14 days	Warning	Slack + email
Audit log write failure	Audit write failure rate > 1% for 5 min	Critical	PagerDuty + Slack

Alert Routing:

Critical → PagerDuty (on-call SRE) + Slack #alerts
Warning → Slack #alerts
Info → Slack #alerts (throttled)

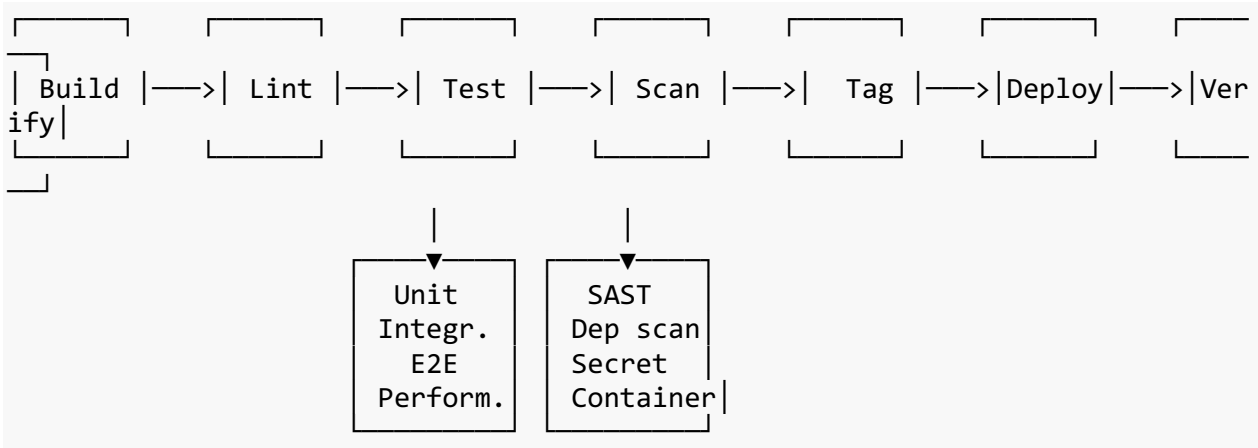
1.22.5 19.5 Dashboards

Dashboard	Audience	Panels
API Overview	Engineering + SRE	Request rate, latency p50/p95/p99, error rate, status code distribution
Business Overview	Product + Engineering	Active auctions, bids/min, winners/hour, revenue, top auctions
Database Health	SRE + DBA	Connection pool, query latency, slow queries, deadlocks,

Dashboard	Audience	Panels
Redis Health	SRE	replication lag Hit ratio, ops/sec, memory, evictions, pub/sub channels
Payment Gateway	Engineering + Finance	Payment success rate, gateway latency, circuit breaker state, webhook processing
Infrastructure	SRE	CPU/memory per pod, pod count, HPA status, network I/O

1.23 20. CI/CD Pipeline Details

1.23.1 20.1 Pipeline Overview



1.23.2 20.2 Pipeline Stages

Stage	Actions	Gate	Duration Target
1. Build	Install deps, npx tsc --noEmit, npm run build (all 5 projects)	Fail on compile error	< 3 min
2. Lint	ESLint + Prettier check on all projects	Fail on lint error	< 1 min
3. Test	Unit tests (coverage gate ≥ 80%), integration	Fail on test failure or coverage	< 10 min

Stage	Actions	Gate	Duration Target
	tests (Testcontainers), E2E tests	drop	
4. Scan	SAST (SonarQube/Semg rep), dependency audit (Snyk), secret scan (Gitleaks), container scan (Trivy)	Fail on critical/high findings	< 5 min
5. Tag	Semantic version tag, build Docker images, push to ghcr.io	Fail on push error	< 5 min
6. Deploy (staging)	Apply Prisma migrations, rolling update to staging Kubernetes	Fail on health check failure	< 5 min
7. Verify (staging)	Smoke tests, DAST scan (ZAP baseline), performance sanity check	Fail on error rate > 1%	< 5 min
8. Deploy (production)	Manual approval gate, apply Prisma migrations, rolling update to production Kubernetes	Fail on health check failure	< 5 min
9. Verify (production)	Smoke tests, monitoring watch (30 min), auto- rollback on error rate > 5%	Auto- rollback if degraded	30 min watch

1.23.3 20.3 GitHub Actions Workflow (Excerpt)

```
name: CI/CD Pipeline
```

```
on:
```

```
  push:
```

```
    branches: [main, develop]
```

```
  pull_request:
```

```

    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npx tsc --noEmit
      - run: npm run build

  lint:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npx eslint . --max-warnings 0
      - run: npx prettier --check .

  test:
    runs-on: ubuntu-latest
    needs: lint
    services:
      postgres:
        image: postgres:15
        env: { POSTGRES_PASSWORD: test }
        ports: ['5432:5432']
      redis:
        image: redis:7
        ports: ['6379:6379']
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npx prisma migrate deploy
      - run: npm test -- --coverage --coverageThreshold='{ "global": { "lines": 80 } }'

  scan:
    runs-on: ubuntu-latest
    needs: test
    steps:
      - uses: actions/checkout@v4
      - run: npx audit-ci --moderate
      - uses: gitleaks/gitleaks-action@v2
      - uses: aquasecurity/trivy-action@master
        with: { image-ref: ghcr.io/takelow/api:latest, severity: CRITICAL,HIGH }

```

```
H }
```

```
deploy-staging:
  runs-on: ubuntu-latest
  needs: scan
  if: github.ref == 'refs/heads/main'
  steps:
    - run: npx prisma migrate deploy
    - run: kubectl rollout restart deployment/api -n staging
    - run: kubectl rollout status deployment/api -n staging

deploy-production:
  runs-on: ubuntu-latest
  needs: deploy-staging
  environment: production
  steps:
    - run: npx prisma migrate deploy
    - run: kubectl rollout restart deployment/api -n production
    - run: kubectl rollout status deployment/api -n production
```

1.23.4 20.4 Deployment Strategies

Environment	Strategy	Rollback
staging	Rolling update (maxSurge: 1, maxUnavailable: 0)	kubectl rollout undo
production	Rolling update (maxSurge: 1, maxUnavailable: 0) with health gate	kubectl rollout undo + monitoring watch
database	Prisma migrate deploy before code deploy (backward-compatible migrations)	Restore from backup (PITR) if data corruption

1.23.5 20.5 Pipeline Gates & Quality Enforcement

Gate	Condition	Stage
Type check	tsc --noEmit passes	Build
Lint	ESLint + Prettier pass	Lint
Unit coverage	≥ 80% lines, ≥ 75% branches	Test
Integration tests	All pass	Test
E2E tests	All critical paths pass	Test
SAST	No critical/high findings	Scan
Dependency audit	No critical/high vulnerabilities	Scan

Gate	Condition	Stage
Secret scan	No secrets in code	Scan
Container scan	No critical/high CVEs	Scan
Staging smoke	Error rate < 1%	Verify (staging)
Production approval	Manual approval from authorized approver	Deploy (production)

End of TTD/LLD v3.0